SUNWAY UNIVERSITY

Jeffrey Cheah Foundation
Nurturing the Seeds of Wisdom

Lecture 04:
Processes, Threads, Interprocess Communication

OUR VISION: TO BE A WORLD CLASS UNIVERSI

NET3204
Distributed Systems

Chee Sun LIEW cheesunl@sunway.edu.my

Learning Outcome

- At the end of this lecture, you will be able to:
  - Explain the concept of processes and threads
  - Describe and distinguish the interprocess communication

Introduction to Threads

- We build virtual processors in software, on top of physical processors:
  - Processor: Provides a set of instructions along with the capability of automatically executing a series of those instructions.
  - Thread: A minimal software processor in whose context a series of instructions can be executed. Saving a thread context implies stopping the current execution and saving all the data needed to continue the execution at a later stage.
  - Process: A software processor in whose context one or more threads may be executed. Executing a thread, means executing a series of instructions in the context of that thread.

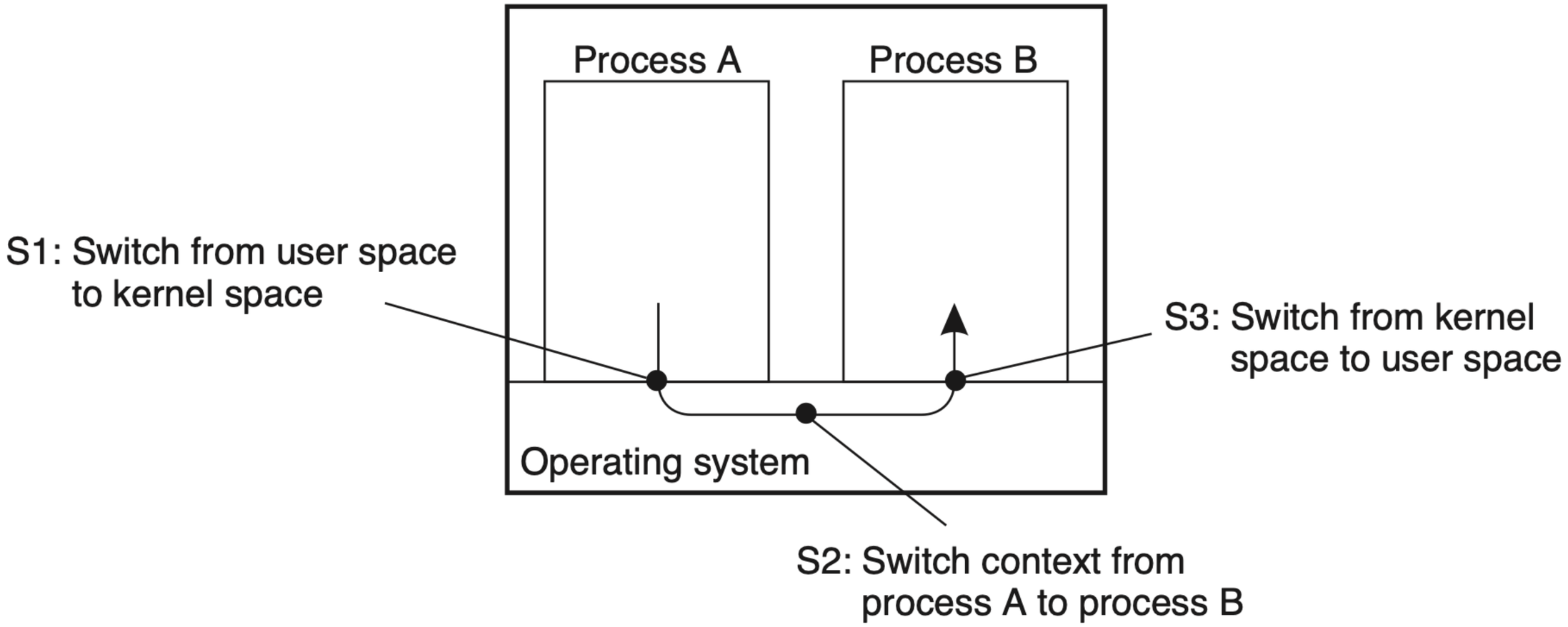
Example of threads vs. processes

- For a computer that is running a spreadsheet program and a word processor:
  - both spreadsheet and word processor are separated processes,
  - within the spreadsheet that comprises many cells with functional dependencies among each other, both recalculation of cells value and interaction with user are separated threads, and
  - within the word processor, both spelling checker and user input handling are separated threads.

Context switching

- Processor context: The minimal collection of values stored in the registers of a processor used for the execution of a series of instructions (e.g., stack pointer, addressing registers, program counter).
- Thread context: The minimal collection of values stored in registers and memory, used for the execution of a series of instructions (i.e., processor context, state).
- Process context: The minimal collection of values stored in registers and memory, used for the execution of a thread (i.e., thread context, but now also at least MMU register values).

Context switching as the result of IPC



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Context switching

- Threads share the same address space. Thread context switching can be done entirely independent of the operating system.
- Process switching is generally more expensive as it involves getting the OS in the loop, i.e., trapping to the kernel.
- Creating and destroying threads is much cheaper than doing so for processes.

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Threads and Operating Systems

- Should an OS kernel provide threads, or should they be implemented as user-level packages?
  - User-space solution
    - All operations can be completely handled within a single process
      - implementations can be extremely efficient.
    - All services provided by the kernel are done on behalf of the process in which a thread resides
      - if the kernel decides to block a thread, the entire process will be blocked.
    - Threads are used when there are lots of external events: threads block on a per-event basis
      - if the kernel can't distinguish threads, how can it support signalling events to them?

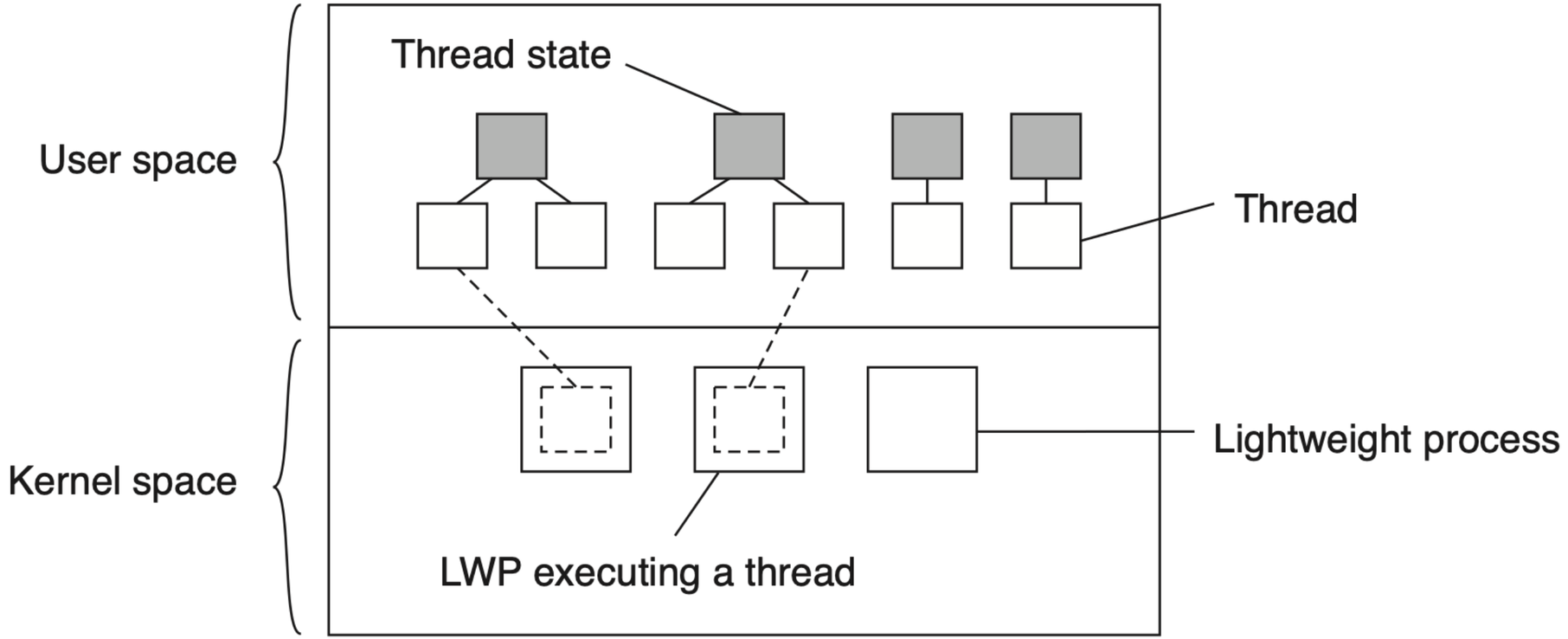
Threads and Operating Systems

- Kernel solution
  - The whole idea is to have the kernel contain the implementation of a thread package. This means that all operations return as system calls
    - Operations that block a thread are no longer a problem: the kernel schedules another available thread within the same process.
    - Handling external events is simple: the kernel (which catches all events) schedules the thread associated with the event.
    - The big problem is the loss of efficiency due to the fact that each thread operation requires a trap to the kernel.
- Conclusion
  - Try to mix user-level and kernel-level threads into a single concept.

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Solaris Threads

- Introduce a two-level threading approach: lightweight processes that can execute user-level threads.
- Combining kernel-level lightweight processes and user-level threads.



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Solaris Threads

- User-level thread does system call
  - the LWP that is executing that thread, blocks.
- The thread remains bound to the LWP.
- The kernel can schedule another LWP having a runnable thread bound to it.
  - Note: this thread can switch to any other runnable thread currently in user space.
- A thread calls a blocking user-level operation
  - do context switch to a runnable thread, (then bound to the same LWP).
- When there are no threads to schedule, an LWP may remain idle, and may even be removed (destroyed) by the kernel.

Threads and Distributed Systems

- Multithreaded Web client
  - Hiding network latencies:
    - Web browser scans an incoming HTML page, and finds that more files need to be fetched.
    - Each file is fetched by a separate thread, each doing a (blocking) HTTP request.
    - As files come in, the browser displays them.

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

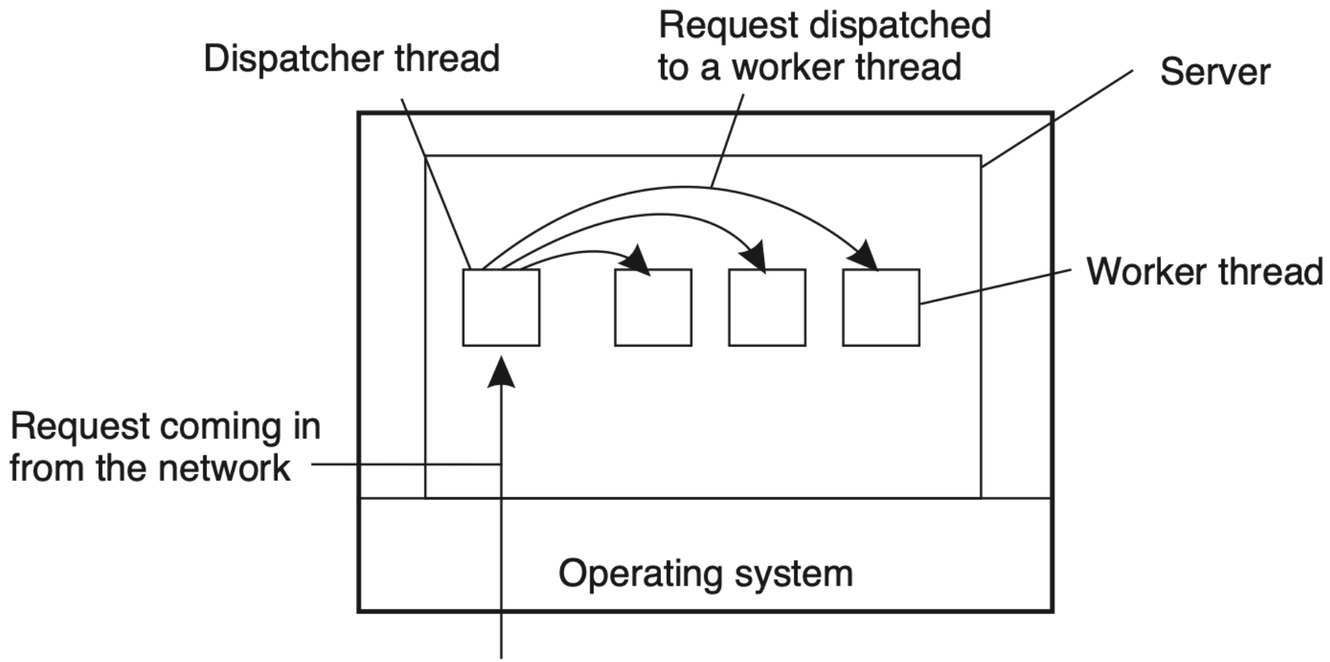
Threads and Distributed Systems

- Multiple request-response calls to other machines (RPC)
  - A client does several calls at the same time, each one by a different thread.
  - It then waits until all results have been returned.
  - Note: if calls are to different servers, we may have a linear speed-up.

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Threads and Distributed Systems

- Multi-threaded servers organised as dispatcher/worker model
  - dispatcher reads incoming request for file operations
  - an idle worker received the request from dispatcher and performs a blocking read from the local file system



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Threads and Distributed Systems

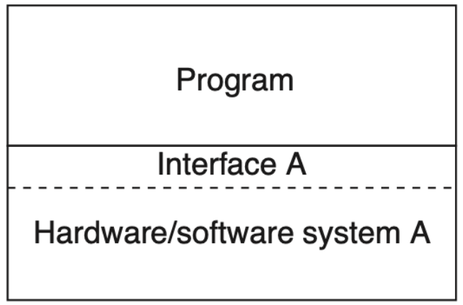
- Three ways to organise the previous file server:
  - threads (parallelism, blocking system calls)
    - dispatcher/worker model
  - single-thread process (non-parallelism, blocking system calls)
    - main loop handles request, performs read, before getting the subsequent requests, in blocking manner
  - finite-state machine (parallelism, non-blocking system calls)
    - single thread with execution states been recorded
    - thread reads a message (e.g. file request, or call returned from disk), processes and stores the state of computation

Threads and Distributed Systems

- Improve performance
  - Starting a thread is much cheaper than starting a new process.
  - Having a single-threaded server prohibits simple scale-up to a multiprocessor system.
  - As with clients: hide network latency by reacting to next request while previous one is being replied.
- Better structure
  - Most servers have high I/O demands. Using simple, well-understood blocking calls simplifies the overall structure.
  - Multithreaded programs tend to be smaller and easier to understand due to simplified flow of control.

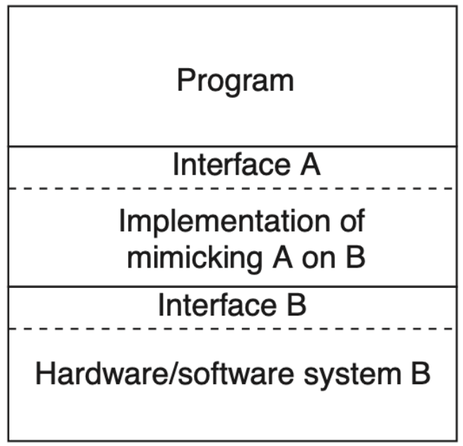
Virtualisation

- Virtualisation is becoming increasingly important:
  - Hardware changes faster than software
  - Ease of portability and code migration
  - Isolation of failing or attacked components



(a)

General organisation between a program, interface, and system

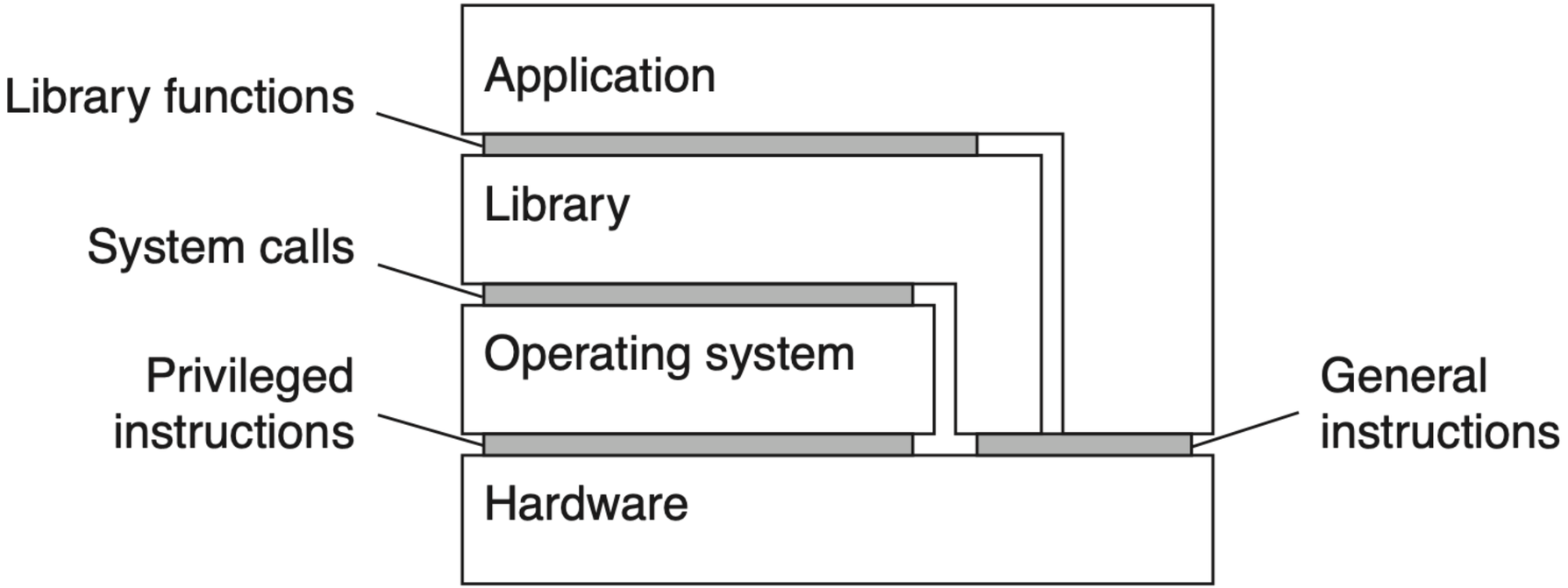


(b)

General organisation of virtualising system A on top of system B

Architectures of Virtual Machines

- Virtualisation can take place at very different levels, strongly depending on the interfaces as offered by various systems components:

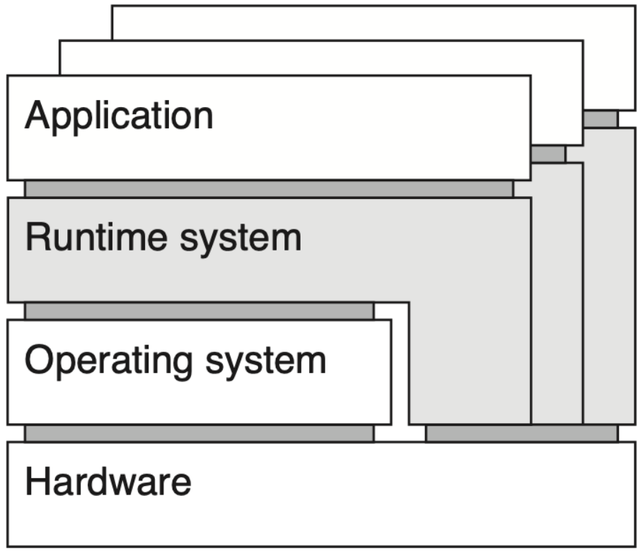


Architectures of Virtual Machines

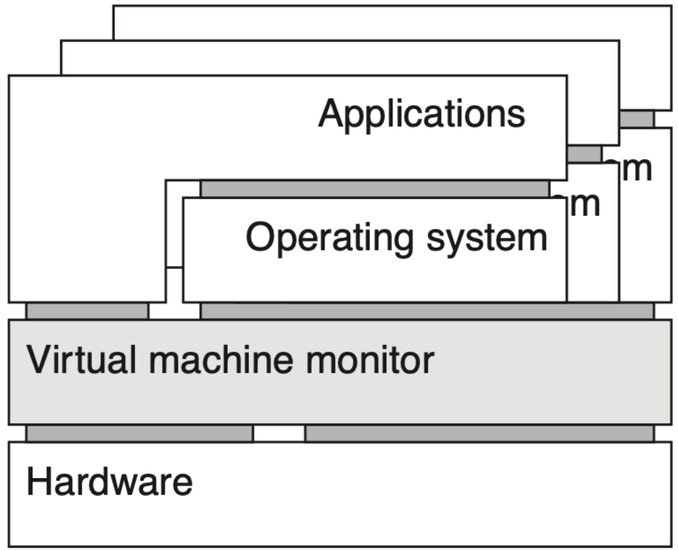
- Interfaces at different levels
  - An interface between the hardware and software consisting of machine instructions
    - that can be invoked by any program.
  - An interface between the hardware and software, consisting of machine instructions
    - that can be invoked only by privileged programs, such as an operating system.
  - An interface consisting of system calls as offered by an operating system.
  - An interface consisting of library calls
    - generally forming what is known as an application programming interface (API).

Process VMs versus VM Monitors

- Process VM: A program is compiled to intermediate (portable) code, which is then executed by a runtime system (Example: Java VM).
- VM Monitor: A separate software layer mimics the instruction set of hardware
  - a complete operating system and its applications can be supported (Example: VMware, VirtualBox).



(a)



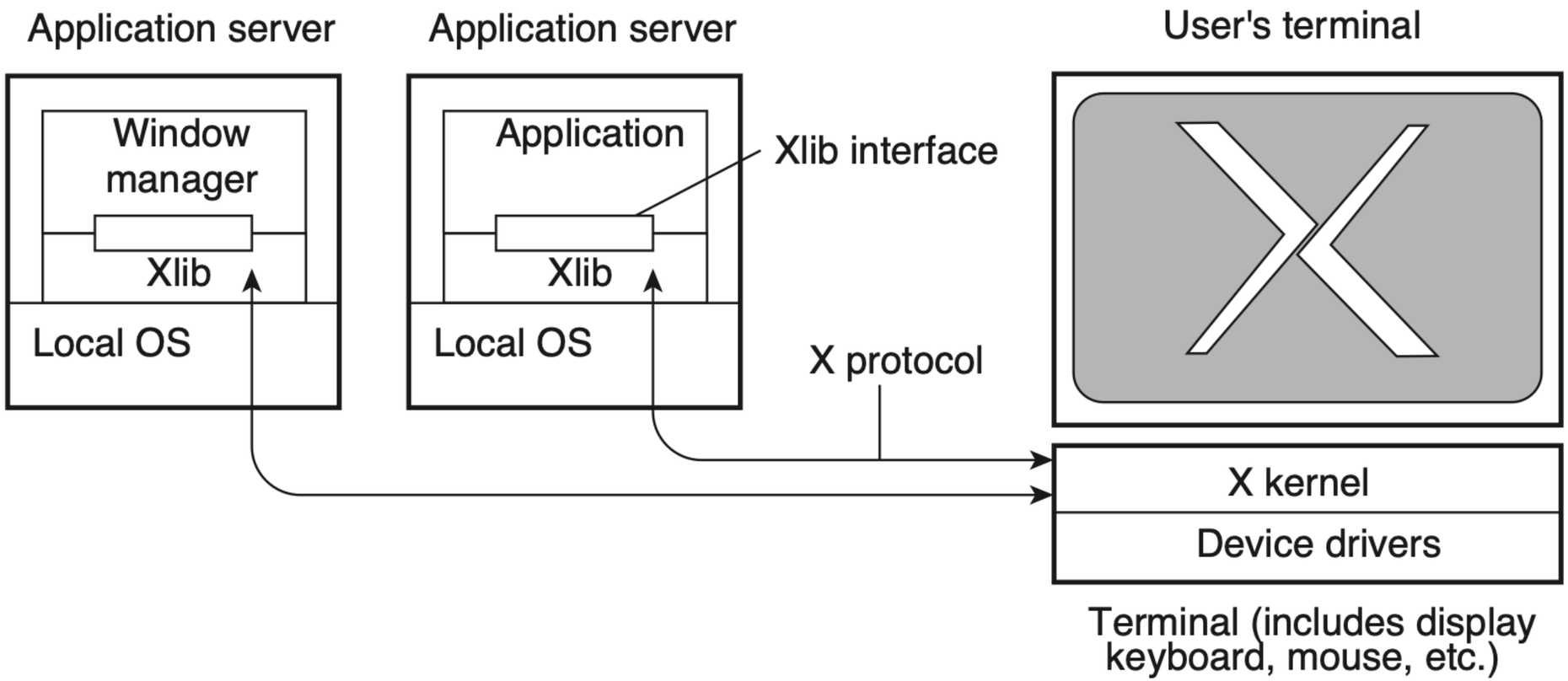
(b)

VM Monitors on operating systems

- We’re seeing VMMs run on top of existing operating systems.
  - Perform binary translation: while executing an application or operating system, translate instructions to that of the underlying machine.
  - Distinguish sensitive instructions: traps to the original kernel (think of system calls, or privileged instructions).
  - Sensitive instructions are replaced with calls to the VMM.

Clients: User Interfaces

- A major part of client-side software is focused on (graphical) user interfaces.



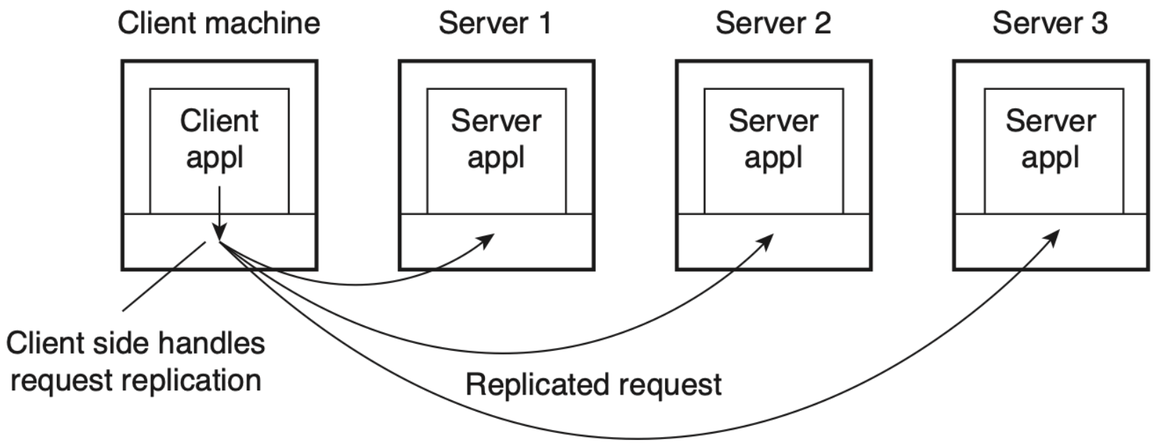
OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Clients: User Interfaces

- User interface is application-aware (inter-application communication):
  - drag-and-drop:
    - move objects across the screen to invoke interaction with other applications
  - in-place editing:
    - integrate several applications at user-interface level (word processing + drawing facilities)

Client-Side Software

- Generally tailored for distribution transparency
  - access transparency: client-side stubs for RPCs
  - location/migration transparency: let client-side software keep track of actual location
  - replication transparency: multiple invocations handled by client stub:



  - failure transparency: can often be placed only at client (we're trying to mask server and communication failures).

Servers: General organisation

- A server is a process that waits for incoming service requests at a specific transport address. In practice, there is a one-to-one mapping between a port and a service.

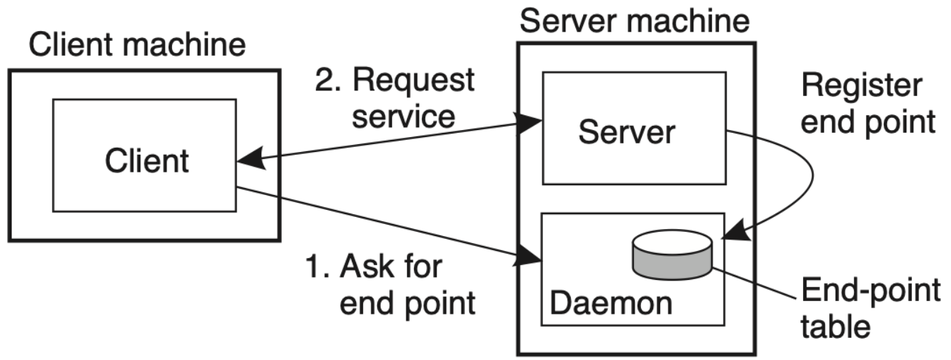
ftp-data	20	File Transfer [Default Data]
ftp	21	File Transfer [Control]
telnet	23	Telnet
	24	any private mail system
smtp	25	Simple Mail Transfer
login	49	Login Host Protocol
sunrpc	111	SUN RPC (portmapper)
courier	530	Xerox RPC

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

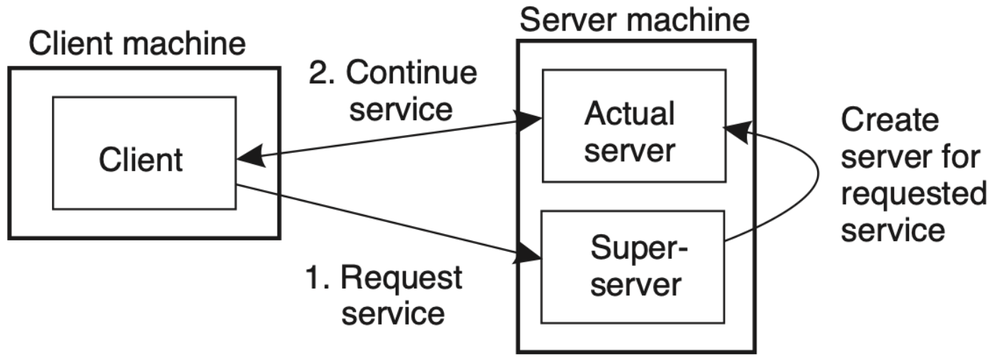
Servers: General organisation

- Type of servers
  - Superservers: Servers that listen to several ports, i.e., provide several independent services. In practice, when a service request comes in, they start a subprocess to handle the request (UNIX inetd)
  - Iterative vs. concurrent servers: Iterative servers can handle only one client at a time, in contrast to concurrent servers

Servers: General design issues



- Client-to-server binding using a daemon (a)



- Client-to-server binding using a superserver. (b)

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Out-of-band communication

- Issue
  - Is it possible to interrupt a server once it has accepted (or is in the process of accepting) a service request?
- Solution 1
  - Use a separate port for urgent data:
    - Server has a separate thread/process for urgent messages
    - Urgent message comes in, associated request is put on hold
    - Note: we require OS supports priority-based scheduling
- Solution 2
  - Use out-of-band communication facilities of the transport layer:
    - Example: TCP allows for urgent messages in same connection
    - Urgent messages can be caught using OS signaling techniques

Servers and state

- Stateless servers
  - Never keep accurate information about the status of a client after having handled a request:
    - Don’t record whether a file has been opened (simply close it again after access)
    - Don’t promise to invalidate a client’s cache
    - Don’t keep track of your clients
- Consequences
  - Clients and servers are completely independent
  - State inconsistencies due to client or server crashes are reduced
  - Possible loss of performance because, e.g., a server cannot anticipate client behaviour (think of prefetching file blocks)

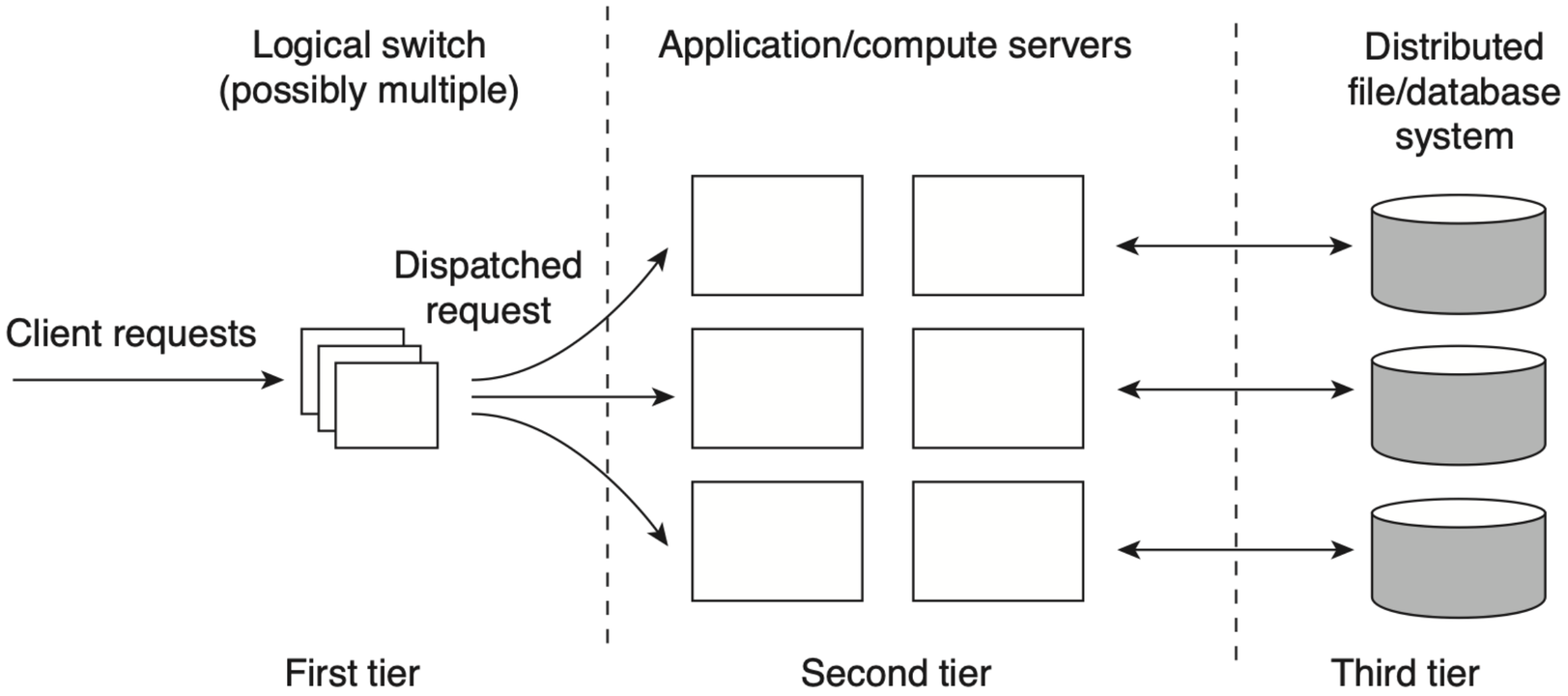
Servers and state

- Stateful servers
  - Keeps track of the status of its clients:
    - Record that a file has been opened, so that prefetching can be done
    - Knows which data a client has cached, and allows clients to keep local copies of shared data
- The performance of stateful servers can be extremely high, provided clients are allowed to keep local copies. As it turns out, reliability is not a major problem.

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Server clusters: three different tiers

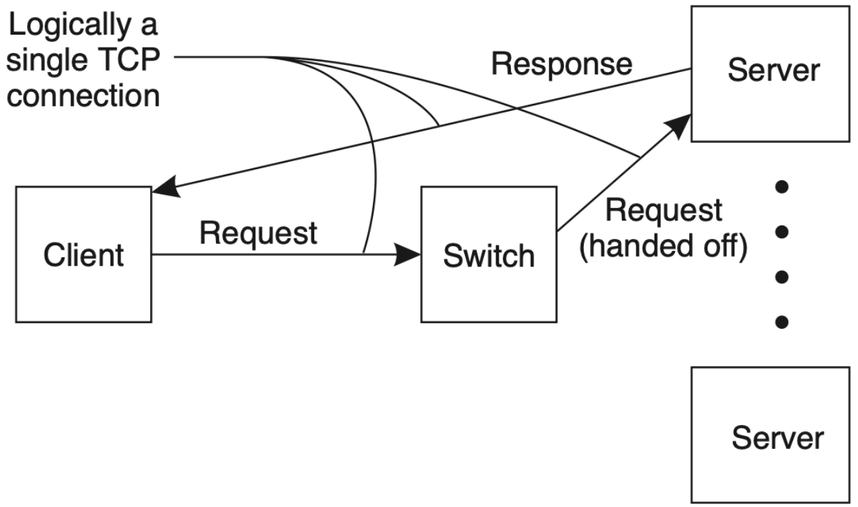
- The first tier is generally responsible for passing requests to an appropriate server.



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Request Handling

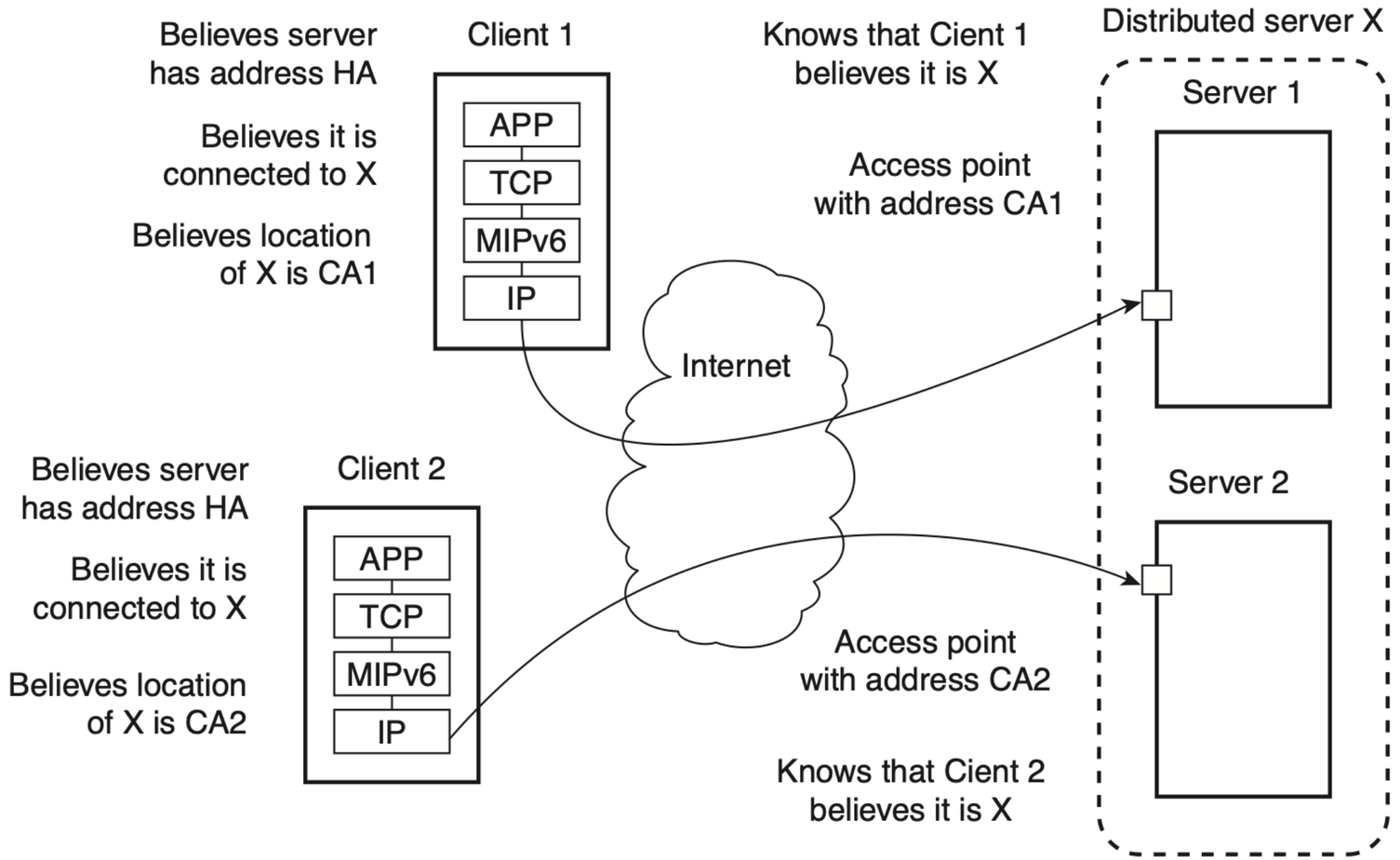
- Having the first tier handle all communication from/to the cluster may lead to a bottleneck.
- Solution
  - Various, but one popular one is TCP-handoff



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Distributed servers

- Route optimisation in a distributed server.



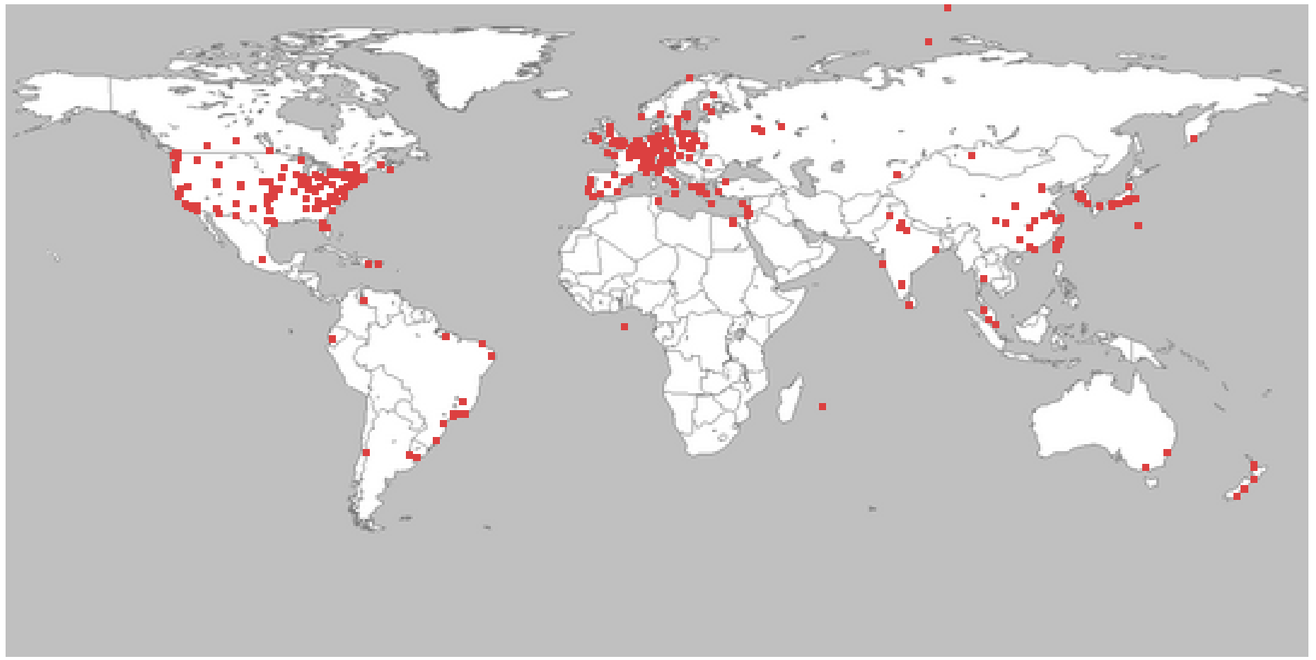
Example: PlanetLab

- Different organisations contribute machines, which they subsequently share for various experiments.
- Problem
  - We need to ensure that different distributed applications do not get into each other's way
    - virtualisation



PlanetLab

- > 1,000 researchers at top academic institutions and industrial research labs have used PlanetLab
  - to develop new technologies for distributed storage, network mapping, peer-to-peer systems, distributed hash tables, and query processing.
- PlanetLab currently consists of 1166 nodes at 551 sites.

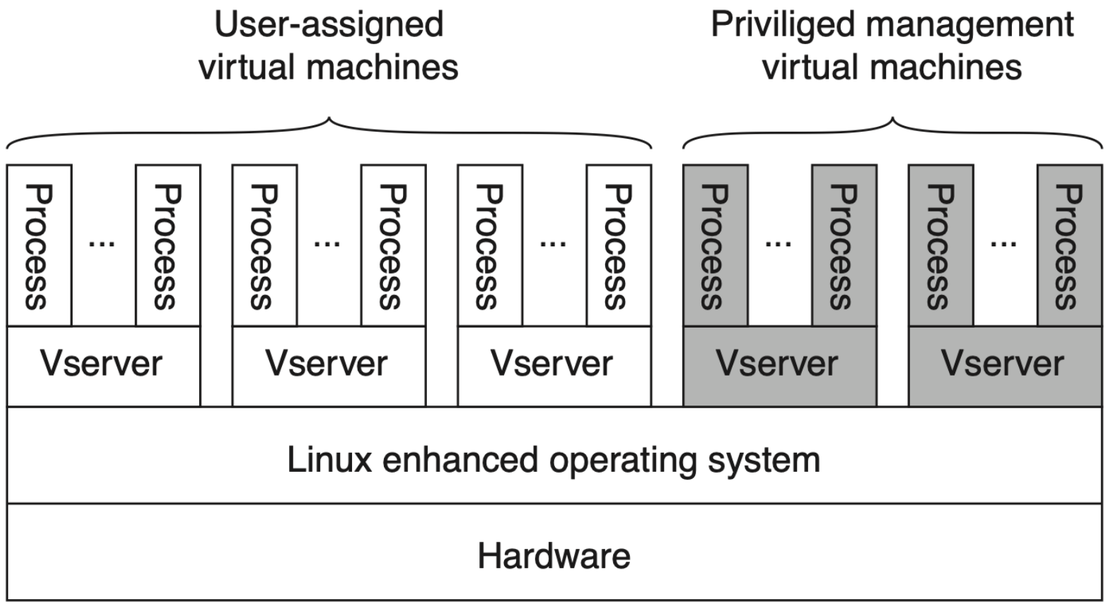


PlanetLab terminology

- Site: A site is a physical location where PlanetLab nodes are located (e.g. Princeton University or HP Labs). Abbreviated versions of site names prefix all slice names.
- Node: A node is a dedicated server that runs components of PlanetLab services.
- Slice: A slice is a set of allocated resources distributed across PlanetLab. To most users, a slice means UNIX shell access to a number of PlanetLab nodes. Slices have a finite lifetime and must be periodically renewed to remain valid.
- Sliver: A set of allocated resources on a single PlanetLab node.

PlanetLab terminology

- Vserver: Independent and protected environment with its own libraries, server versions, and so on. Distributed applications are assigned a collection of vservers distributed across multiple machines (slice).

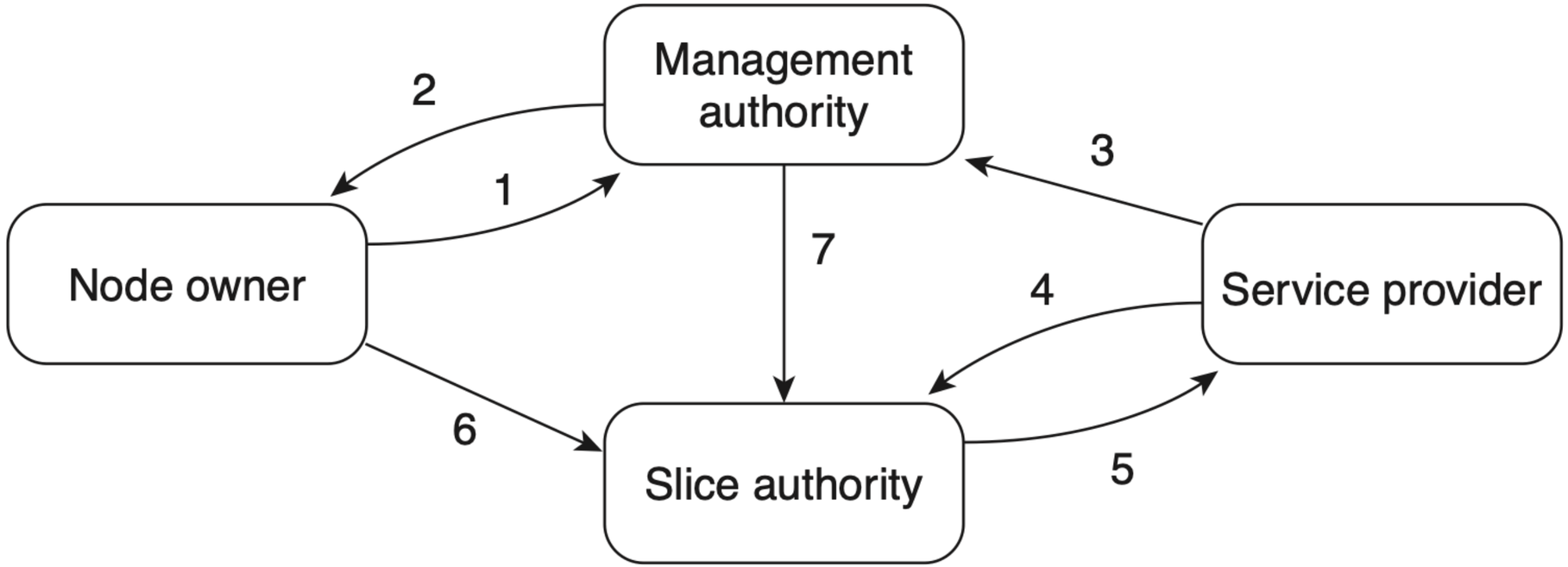


PlanetLab

- PlanetLab management issues:
  - Nodes belong to different organisations.
    - Each organisation should be allowed to specify who is allowed to run applications on their nodes,
    - And restrict resource usage appropriately.
  - Monitoring tools available assume a very specific combination of hardware and software.
    - All tailored to be used within a single organisation.
  - Programs from different slices but running on the same node should not interfere with each other.

PlanetLab

- The management relationships between various PlanetLab entities



PlanetLab

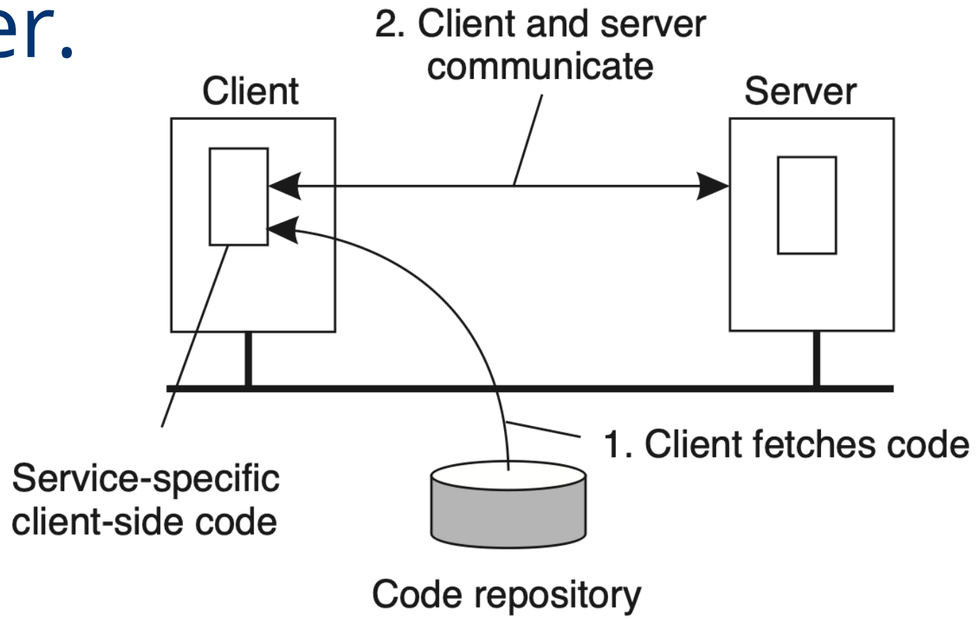
- Relationships between PlanetLab entities:
  - A node owner puts its node under the regime of a management authority, possibly restricting usage where appropriate.
  - A management authority provides the necessary software to add a node to PlanetLab.
  - A service provider registers itself with a management authority, trusting it to provide well-behaving nodes.

PlanetLab

- Relationships between PlanetLab entities:
  - A service provider contacts a slice authority to create a slice on a collection of nodes.
  - The slice authority needs to authenticate the service provider.
  - A node owner provides a slice creation service for a slice authority to create slices. It essentially delegates resource management to the slice authority.
  - A management authority delegates the creation of slices to a slice authority.

Reasons for Migrating Code

- Performance!!!
  - load distribution, parallelism,
- The client first fetches the necessary software, and then invokes the server.

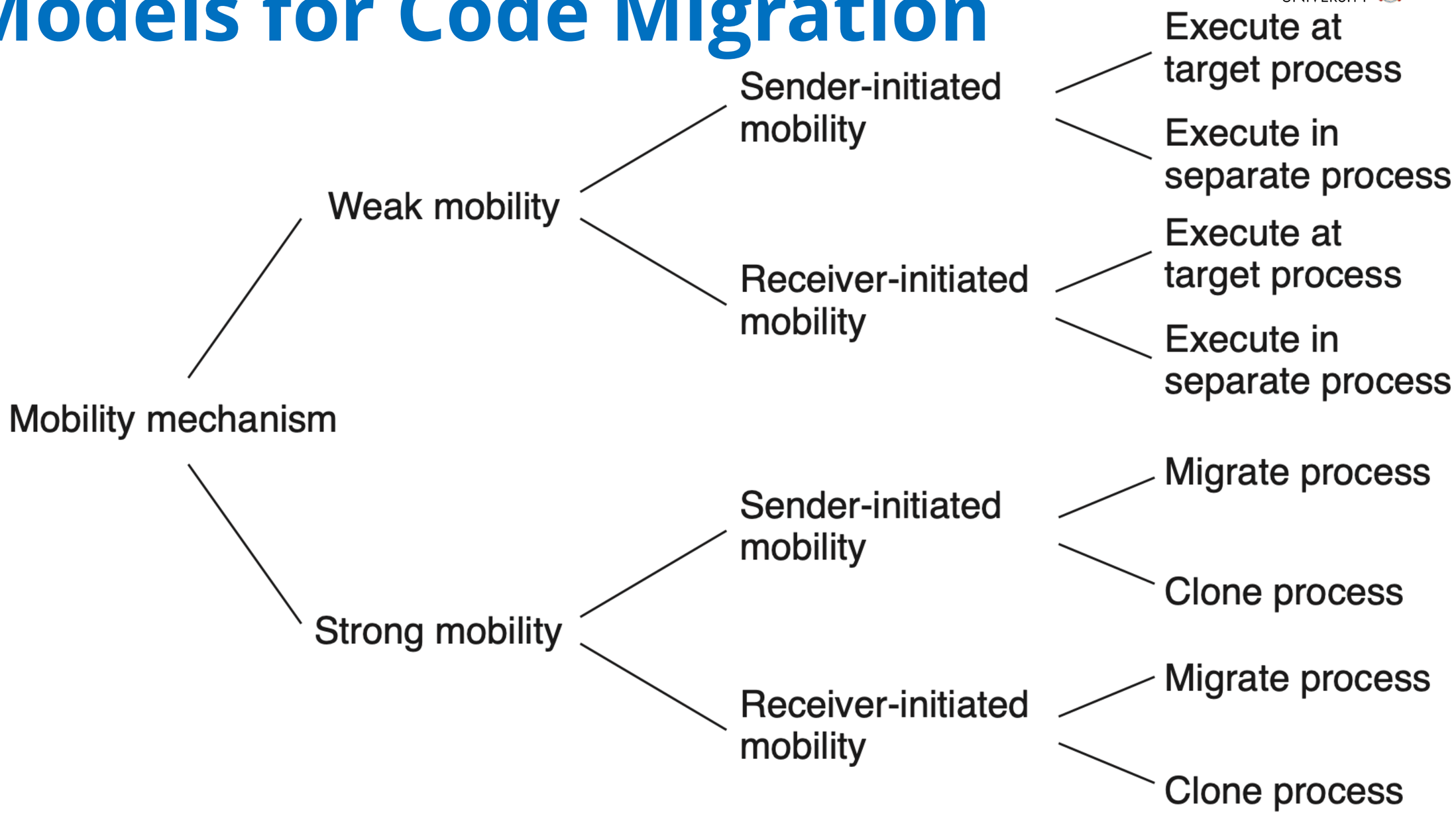


OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Strong and weak mobility

- Object components
  - Code segment: contains the actual code
  - Data segment: contains the state
  - Execution state: contains context of thread executing the object's code
- Weak mobility
  - Move only code and data segment (and reboot execution):
    - Relatively simple, especially if code is portable
    - Distinguish code shipping (push) from code fetching (pull)
- Strong mobility
  - Move component, including execution state
    - Migration: move entire object from one machine to the other
    - Cloning: start a clone, and set it in the same execution state.

Models for Code Migration



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Managing local resources

- Problem
  - An object uses local resources that may or may not be available at the target site.
- Resource types
  - Fixed: the resource cannot be migrated, such as local hardware (e.g. local drive)
  - Fastened: the resource can, in principle, be migrated but only at high cost (e.g. local database and web site)
  - Unattached: the resource can easily be moved along with the object (e.g. a data files associated with program)

Managing local resources

- Process-to-resource binding
  - By identifier: the object requires a specific instance of a resource (e.g. a specific database, URL)
  - By value: the object requires the value of a resource (e.g. local binding of libraries in programming language)
  - By type: the object requires that only a type of resource is available (e.g. a colour monitor)

Managing Local Resources

Process-to-resource binding

Resource-to-machine binding

	Unattached	Fastened	Fixed
ID	MV (or GR)	GR (or MV)	GR
Value	CP (or MV, GR)	GR (or CP)	GR
Type	RB (or MV, GR)	RB (or GR, CP)	RB (or GR)

GR = Establish global systemwide reference

MV = Move the resource

CP = Copy the value of the resource

RB = Re-bind to a locally available resource

Migration in Heterogeneous Systems

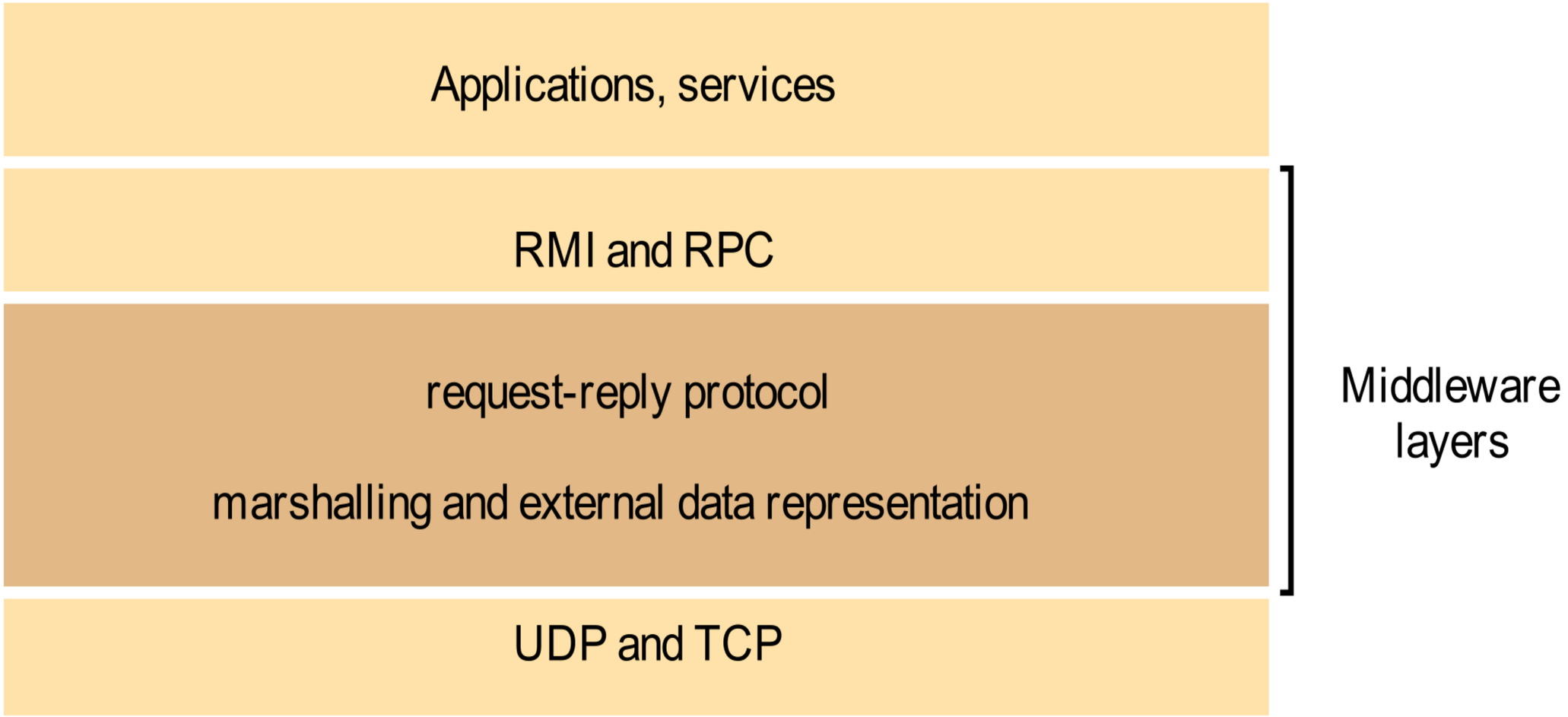
- Three ways to handle migration (which can be combined)
  - Pushing memory pages to the new machine and resending the ones that are later modified during the migration process.
  - Stopping the current virtual machine; migrate memory, and start the new virtual machine.
  - Letting the new virtual machine pull in new pages as needed, that is, let processes start on the new virtual machine immediately and copy memory pages on demand.

Migration in heterogenous systems

- Main problem
  - The target machine may not be suitable to execute the migrated code
  - The definition of process/thread/processor context is highly dependent on local hardware, operating system and runtime system
- Only solution
  - Make use of an abstract machine that is implemented on different platforms:
    - Interpreted languages, effectively having their own VM
    - Virtual VM (as discussed previously)

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Middleware layers



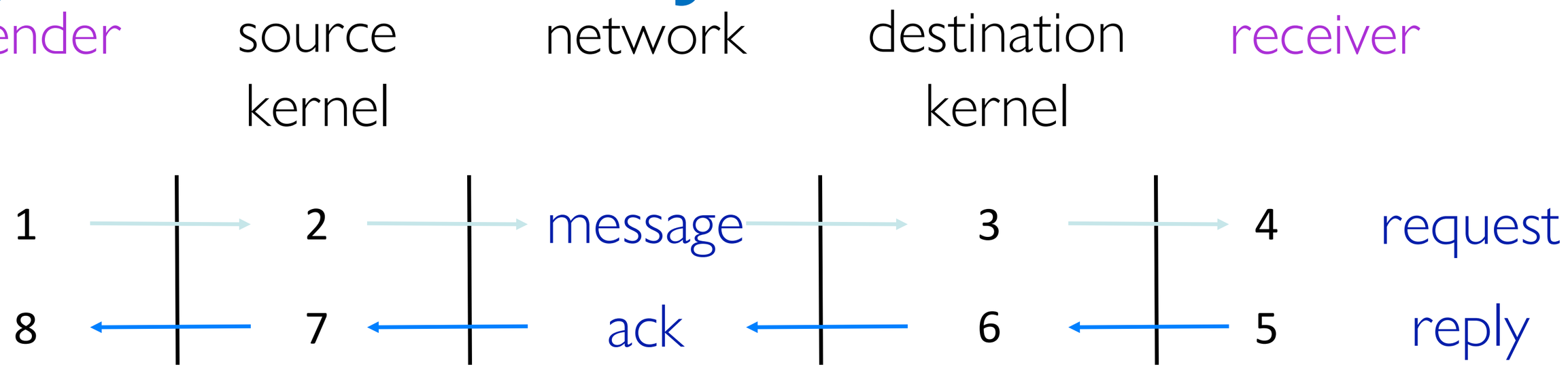
API for Internet Protocol

- API (Application Program Interface) – a set of routines, protocols, and tools for building software applications. A good API makes it easier to develop a program by providing all the building blocks. A programmer puts the blocks together,
- The characteristics of interprocess communication:
  - In order for one process to communicate with another, one process sends a message to a destination and another process at the destination receives the message.

Synchronous vs Asynchronous

- Basic operation
  - `Send (dest, &ptr)`
  - `Receive (source, &ptr)`
- A queue is associated with each message destination, which can be regarded as a buffer between the “message producer" (Sender) and the “message consumer" (Receiver).
  - When the buffer is empty the Receiver must wait;
  - when the buffer is full the Sender must wait, or messages will be lost.

Synchronous vs Asynchronous



- Nonblocking send 1+8
- Blocking send 1+2+7+8
- Reliable blocking send 1+2+3+6+7+8
- Explicit blocking send 1+2+3+4+5+6+7+8
- Request & reply 1-4, service, 5-8

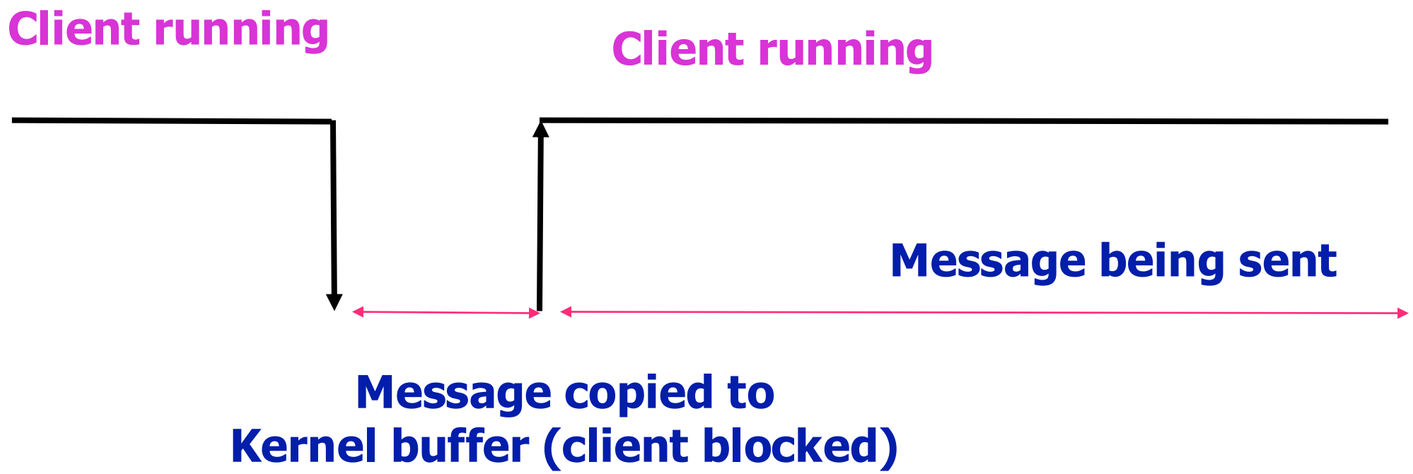
Synchronous vs Asynchronous

- **Synchronous** – sender is blocked until message is completely sent or receiver does not return until the whole message is received.
- **Asynchronous Send ()** – return control to caller immediately
  - Advantage – minimise idle time
  - But must handle with care – message buffer must not be overwritten
    - copied to kernel buffer
- **Asynchronous Receive ()** – inform kernel the address of the buffer and return to caller.
  - Have to use Wait () to check whether the receive has completed.

Synchronous vs Asynchronous



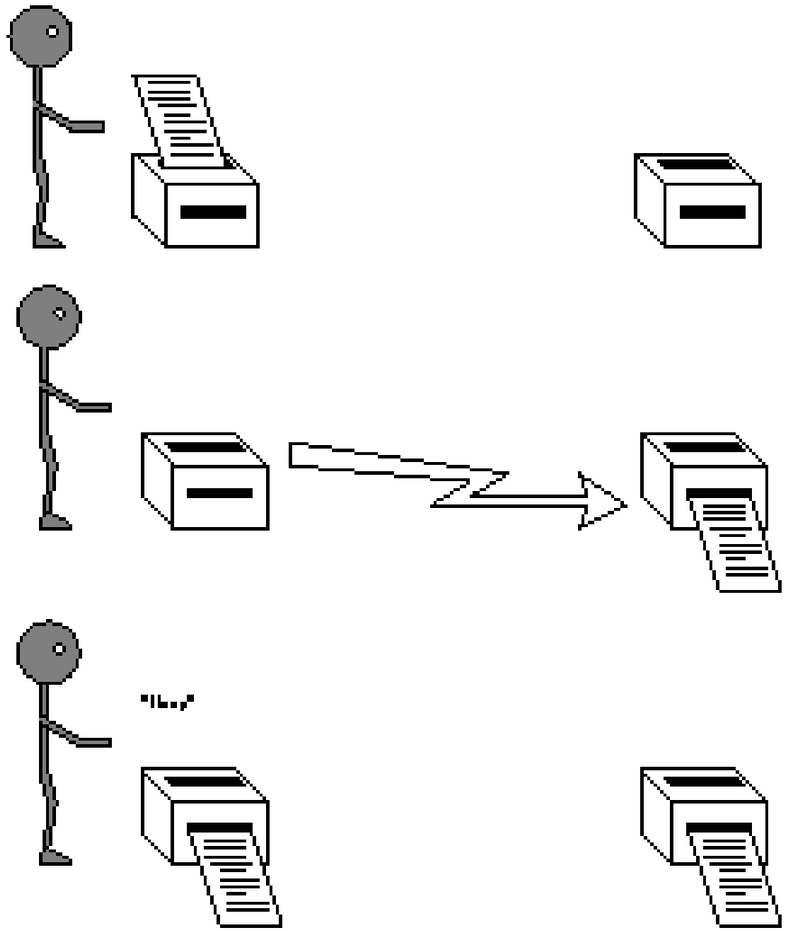
(a) Synchronous



(b) Asynchronous

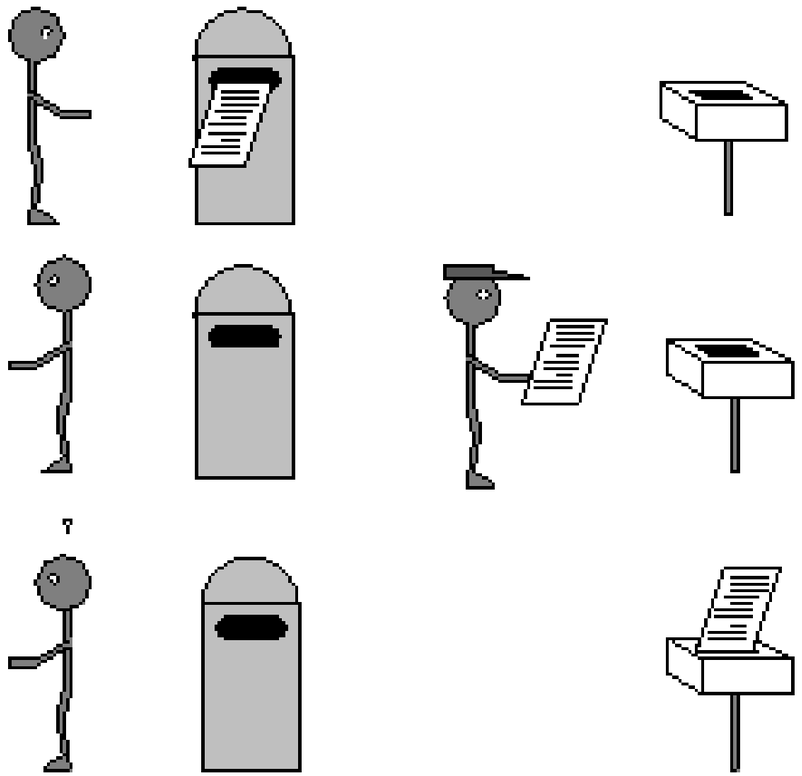
Synchronous vs Asynchronous

- A synchronous communication does not complete until the message has been received
  - A FAX or registered mail



Synchronous vs Asynchronous

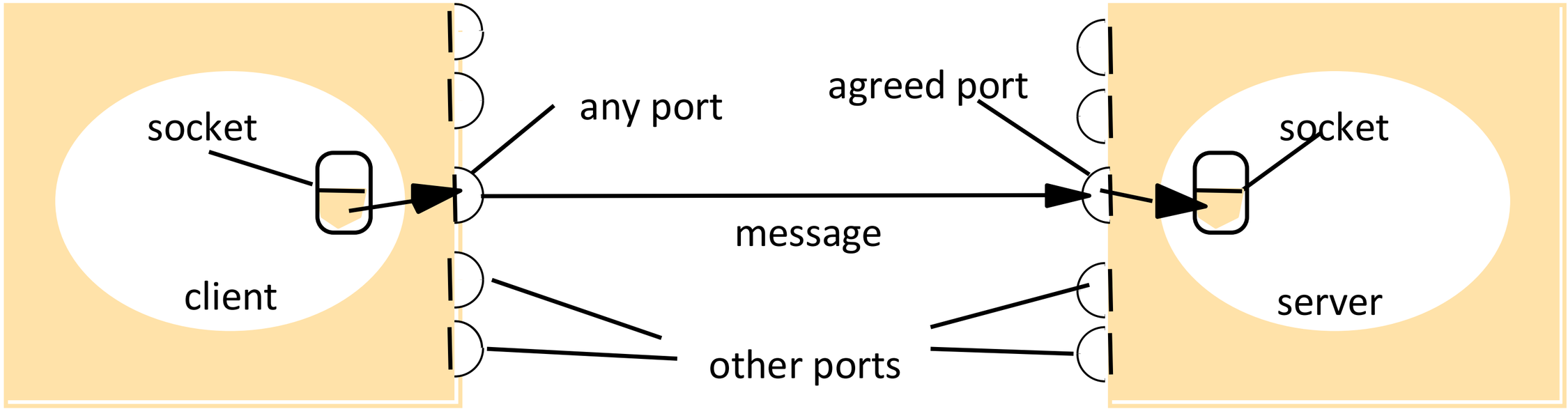
- An asynchronous communication completes as soon as the message is on the way.
  - A post card or email



Sockets and ports

- **Socket** – endpoint for communication between processes
  - IPC – transmitting message between a socket from one process and a socket in another
  - Process use send/receive
    - Create and bind socket to internet address and local port
    - Use the same port for sending and receiving
    - Processes do not share ports (exception to IP multicast)
  - Both UDP and TCP use socket

Sockets and ports (cont.)



Internet address = 138.37.94.248

Internet address = 138.37.88.249

Sockets and ports (cont.)

- Message destinations – local port
  - In the IPs, messages are sent to (**Internet address, local port**) pairs.
  - There are 2^{16} possible port numbers
  - A process **cannot share ports** with other processes on the same computer
  - However, any number of processes may send messages to the same port.

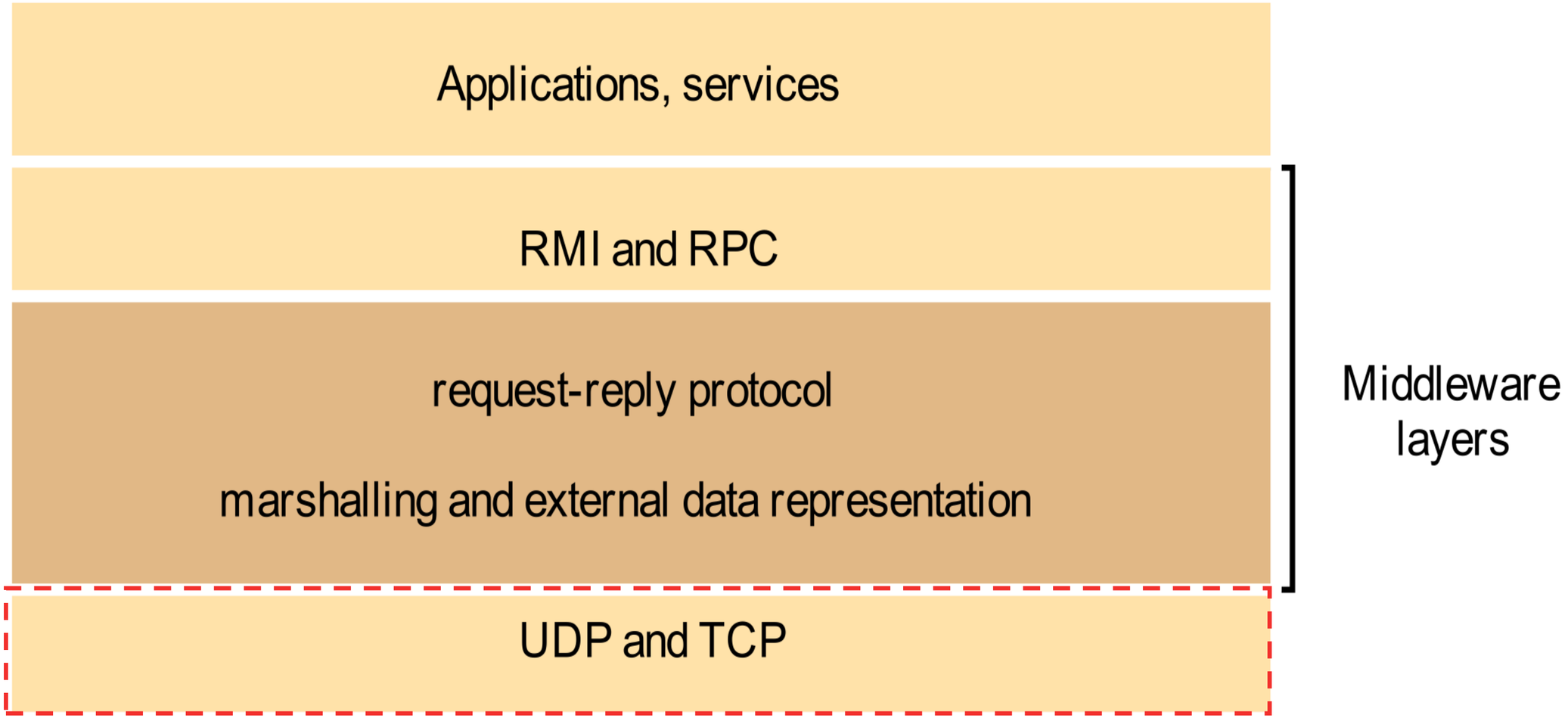
Sockets and ports (cont.)

- What is a local port/port number ?
  - A local port is a message destination within a computer.
  - In a TCP/IP-based network such as the internet, it is a number assigned to an application program running in the computer. The number is used to link the incoming data to the correct service.
  - Well-known ports are standard port numbers used by everyone; for example, port 80 is used for HTTP traffic (Web traffic).
- What if a port has several receivers?
  - If several processes share a port, then all of the messages that arrive on that port must be received and processed independently by those processes.

Sockets and ports (cont.)

- Reliability –
  - validity (any message in the outgoing message buffer is eventually delivered to the incoming message buffer)
  - integrity (uncorrupted and without duplication)
- Ordering –
  - receiver needs to receive messages in the sender order. Such as large packets, which need to be fragmented before being sent.

Middleware layers



UDP and TCP

UDP

(User Datagram Protocol):

- connectionless protocol
- Used when a a reliable delivery is not required. - provides very few error recovery services,
- The application itself must process any errors and check for reliable delivery.

TCP

(Transmission Control Protocol):

- connection-oriented and stream-oriented
- provides for reliable communication over packet-switched networks.

UDP and TCP (cont.)

UDP

- A datagram sent by UDP is transmitted from a sending process to a receiving process without acknowledgement or retries.

TCP

– TCP ensures that a message is sent accurately and in its entirety.

Lost messages:

- If the sender does not receive an acknowledgement within a timeout, it retransmits the message.

UDP and TCP (cont.)

UDP

- Blocking:
  - Uses non-blocking sends and blocking receives.
  - The message on the receiving computer will be put on queue for the socket that is bound to the destination port.

TCP

- Blocking:
  - Block the writer (sender) until the reader (receiver) has consumed sufficient data.
  - The data will be kept in a queue at the destination socket. The process attempts to read data from an input channel, will get data from the queue or it will block until data becomes available.

UDP and TCP (cont.)

UDP

- Message size:
  - The receiving process needs to specify an array of particular size in which to receive a message.
  - The message is truncated if too large.
  - Packet length allowed up to 2^{16} bytes (64 kB). Restriction on some networks to 8 kB.
  - Larger message must be fragmented by the sender.

TCP

- Message sizes:
  - The application can choose how much data it writes to a stream or reads from it.
  - TCP decides how much data to collect before transmitting it as one or more IP packets.

UDP and TCP (cont.)

UDP

- Timeouts:
  - Timeouts can be set on sockets so that the receive operation does not wait indefinitely.
- Receive from any:
  - The receive method does not specify an origin for messages (I.e. can be from any origin addressed to its socket).

TCP

- Message duplication and ordering:
  - Use of message identifiers enable the recipient to detect and reject duplicates, or to reorder messages.
- Threads:
  - When a server accepts a connection, it generally creates a new thread in which to communicate with the new client. What is the advantage?

UDP and TCP (cont.)

TCP (cont…)

- Message destinations:
  - Pre-establish connection by communicating connect request and accept request.
  - Once connection is establish, the processes simply read from and write to the stream without needing to use Internet addresses and ports.

- The API for stream communication assumes that when a pair of processes are establishing a connection, one of them plays the client role and the other plays the server role, but thereafter they could be peers.

UDP and TCP (cont.)

UDP

- Failure Model:
  - Reliable communication definition: validity+integrity.
  - UDP datagrams suffer from the following failures:
    - Omission failures: Messages may be dropped occasionally (checksum error / buffer Full)
    - Ordering: Messages can sometimes be delivered out of sender order.

TCP

- Failure model:
  - use checksum and sequence numbers (integrity)
  - timeouts and retransmission (validity).
  - In the case of broken connection:
    - the processes using the connection cannot distinguish between network failure and failure of the process at the other end of the connection.
    - the communicating processes cannot tell whether their recent messages have been received or not.

UDP and TCP (cont.)

UDP

- Use of UDP:
  - For some applications, it is acceptable to use a service that is liable to occasional omission failures.
  - E.g. DNS.
- Java API for UDP datagrams:
  - The Java API provides datagram communication by means of two classes: DatagramPacket and DatagramSocket.

TCP

- Use of TCP:
  - E.g. HTTP, FTP, Telnet, SMTP.
  - All have reserved port numbers.
- Java API for TCP streams:
  - The Java interface to TCP streams is provided in the classes ServerSocket and Socket.

Socket Used for Datagram

- Both processes create a socket and get a descriptor
- Bind their sockets to socket addresses then communicate

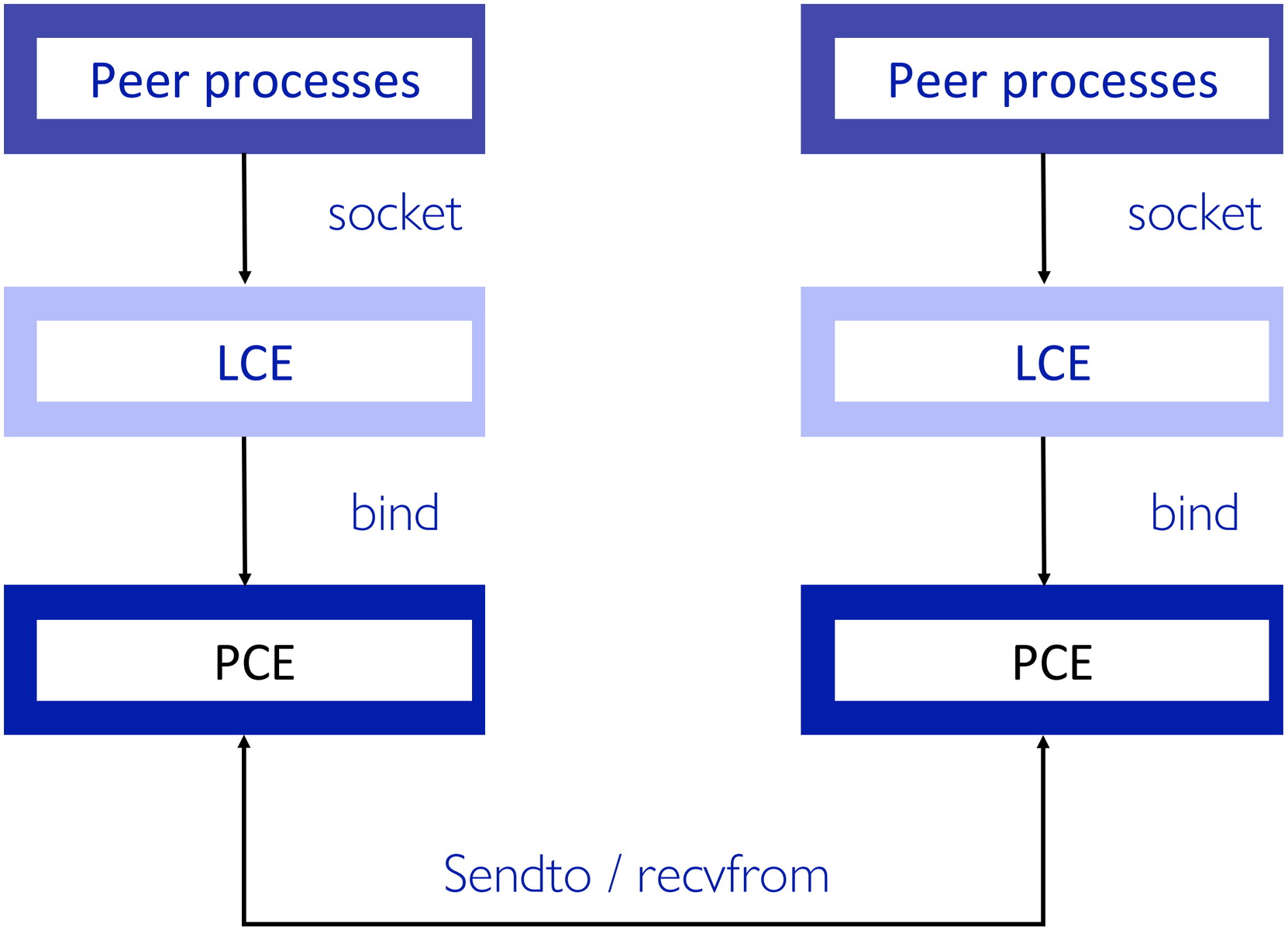
Sending a message

```
s = socket(AF_INET, SOCK_DGRAM, 0)
•
•
bind(s, ClientAddress)
•
•
sendto(s, "message", ServerAddress)
```

Receiving a message

```
s = socket(AF_INET, SOCK_DGRAM, 0)
•
•
bind(s, ServerAddress)
•
•
amount = recvfrom(s, buffer, from)
```

UDP Socket Communication



Socket Used for Datagram (cont.)

- Socket system call – specify the communication domain (normally the Internet), the type (datagram or stream), and sometimes a particular protocol.
- Before a pair of processes can communicate, the recipient must bind its socket address if it requires a reply.

- **sendto** – with arguments specifying the socket through which the message is to be sent, the message, the socket address of destination
- **recvfrom** – specify local socket to receive a message, memory locations for storing, the socket address of the sending socket.

Sockets used for Streams

- Listening process (server) create a stream socket and binds to the server's socket address and wait for request
- Server accepts connection and obtain a new socket for communication with client

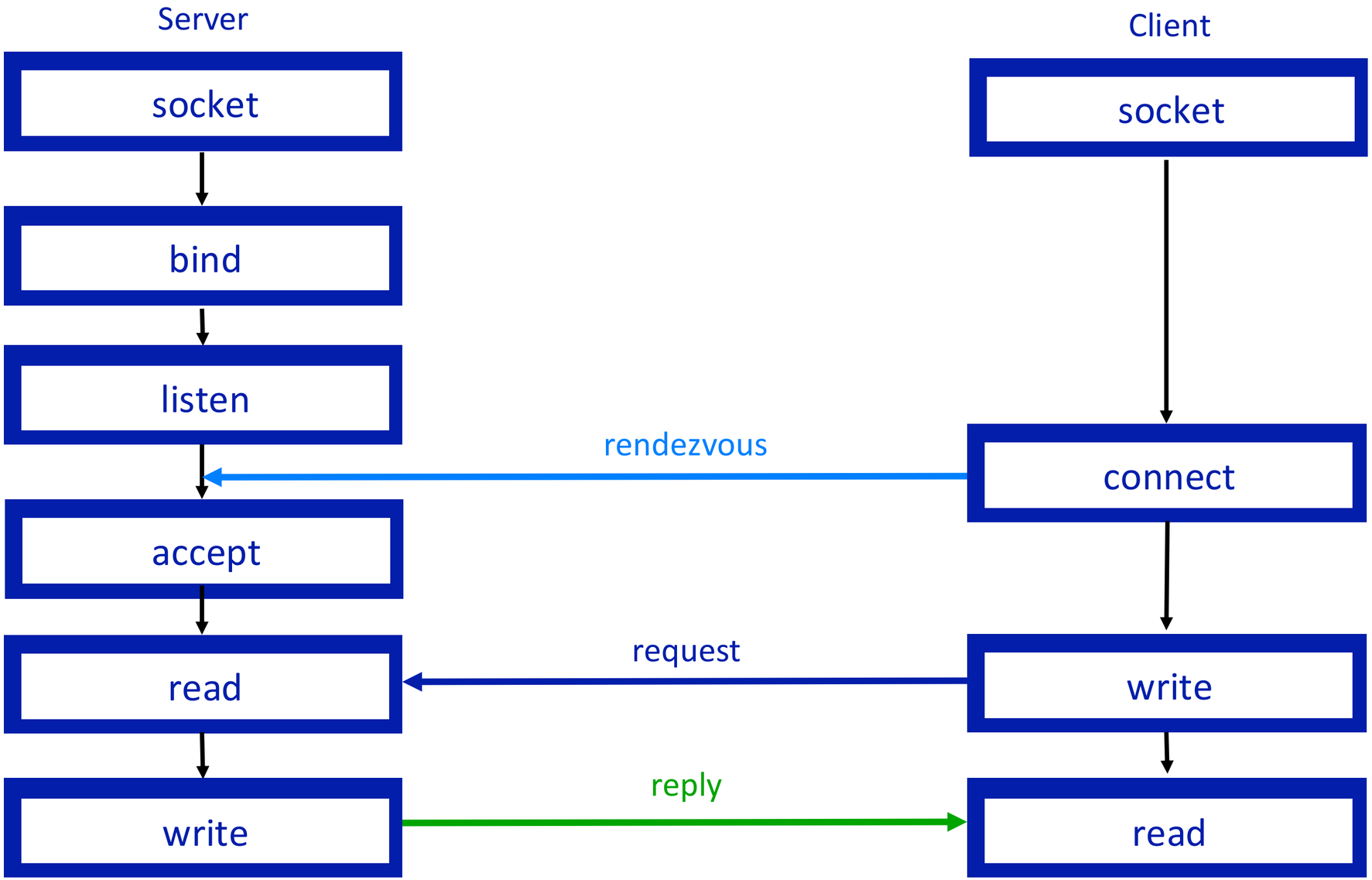
Requesting a connection

```
s = socket(AF_INET, SOCK_STREAM,0)
●
●
connect(s, ServerAddress)
●
●
write(s, "message", length)
```

Listening and accepting a connection

```
s = socket(AF_INET, SOCK_STREAM,0)
●
bind(s, ServerAddress);
listen(s,5);
●
sNew = accept(s, ClientAddress);
●
n = read(sNew, buffer, amount)
```

TCP Client-Server Socket Communication



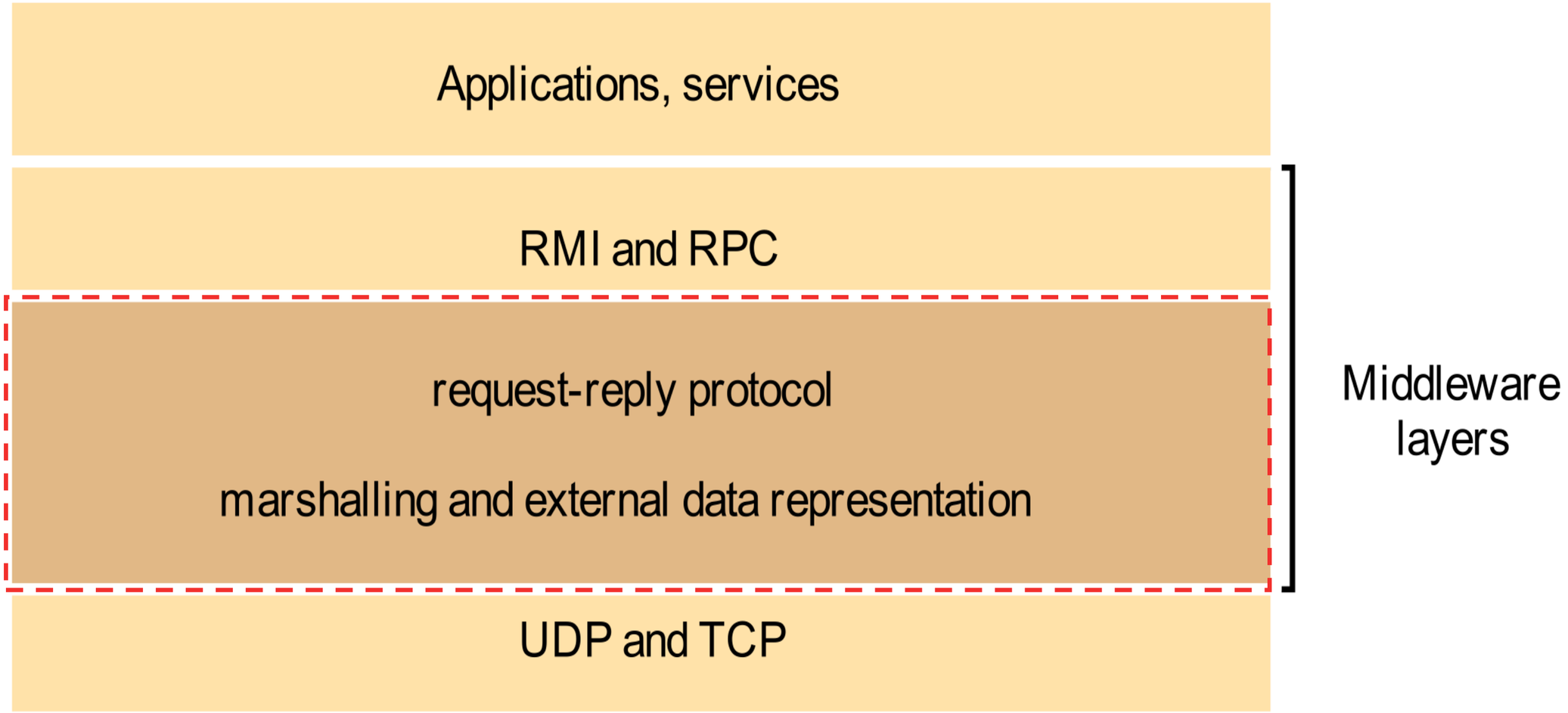
Sockets used for Streams (cont.)

- In order to use the stream protocol, two processes must first establish a connection between their pair of sockets.
- One of the sockets will be listening for a request and the other will be asking for a connection.
- **listen** – listen on its socket for client requests for connections. Also specifies the maximum number of requests for connections that can be queued at this socket.
- **accept** – accept a connection requested by a client and obtains a new socket for communication with that client.
- **connect** – client request connection via the socket address.

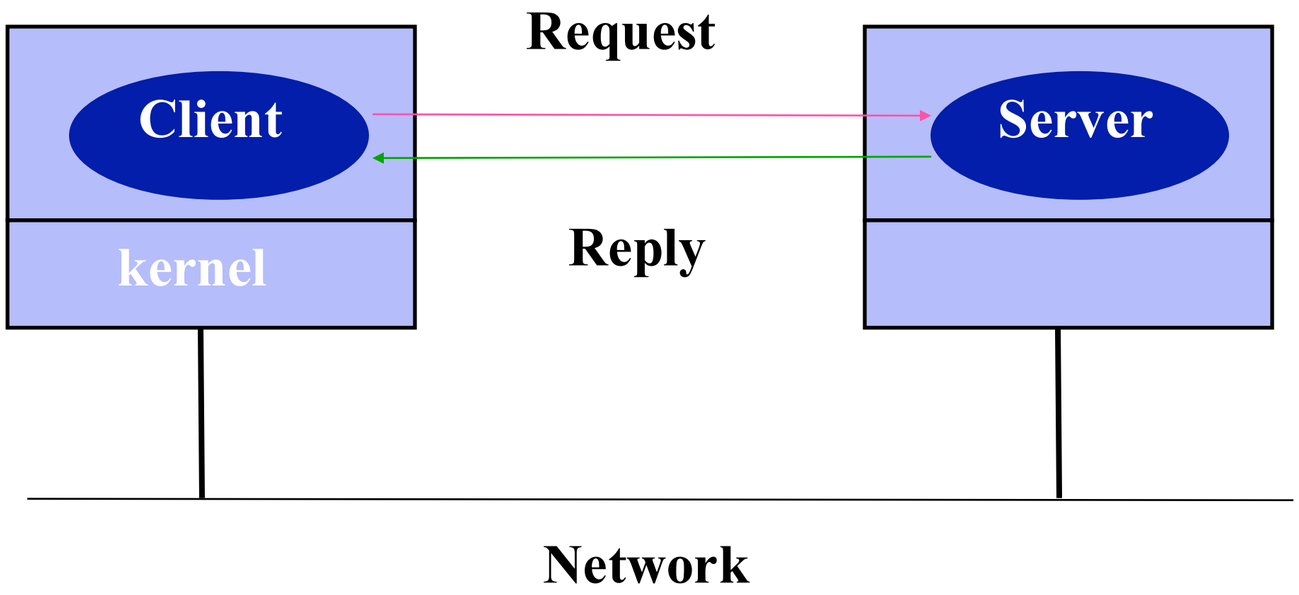
The Client-Server Model

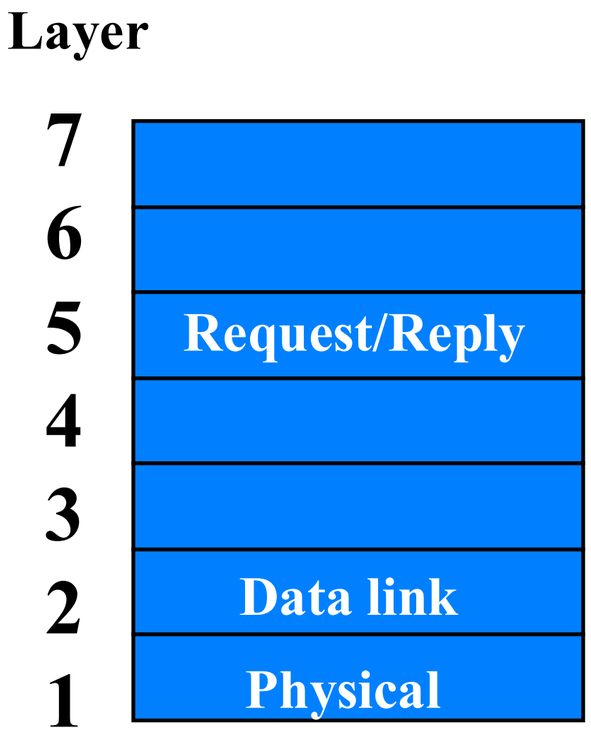
- A group of cooperating process; servers and clients where server provide services
- Based on connectionless protocols
  - Client sends a request message to the server asking for service
  - Server does the work and returns the requested data (or reply / acknowledgement) or error code.
  - No connection is established

Middleware layers



The Client-Server Model (cont.)





The Client-Server Model (example)

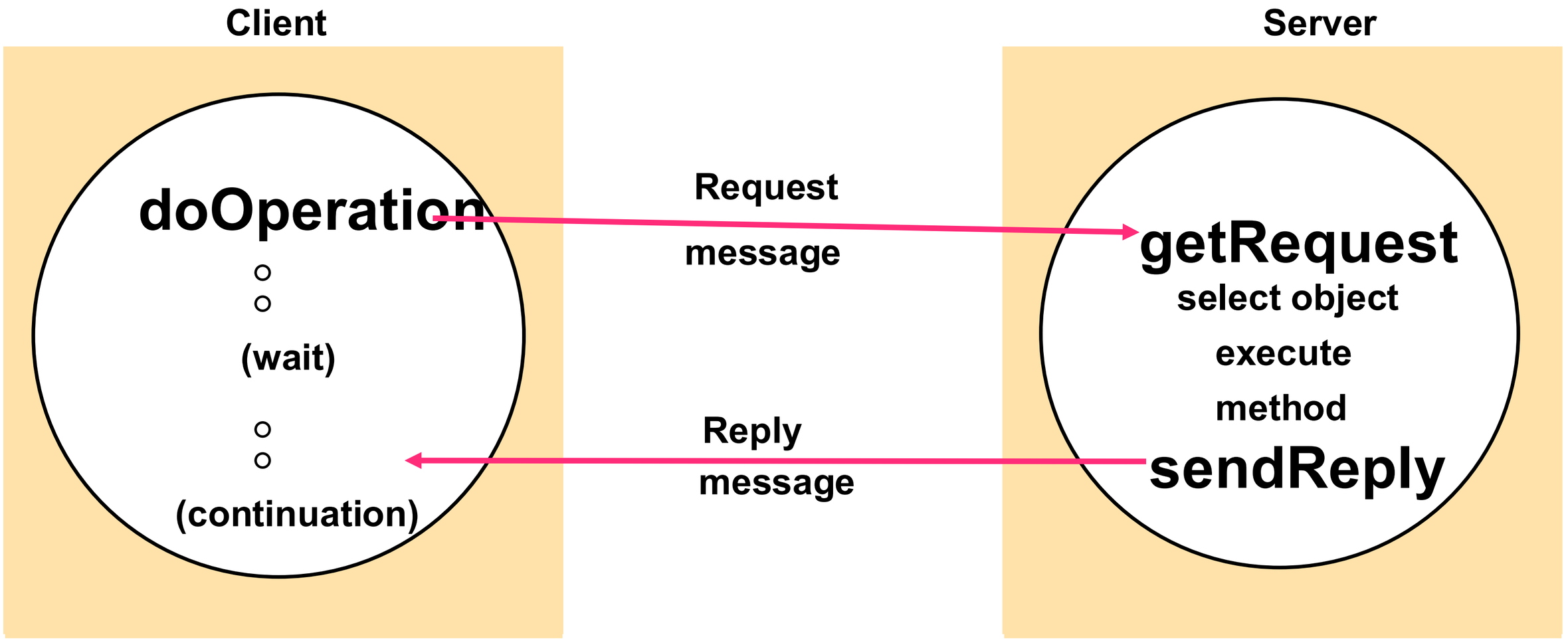
Server

```
While (1) {
    Receive request
    Service the message
}
Send result
```

Client

```
Do {
    Prepare message
    Send message
    Blocking wait to receive
    Process the result
} while (!finished)
```

Request-reply communication



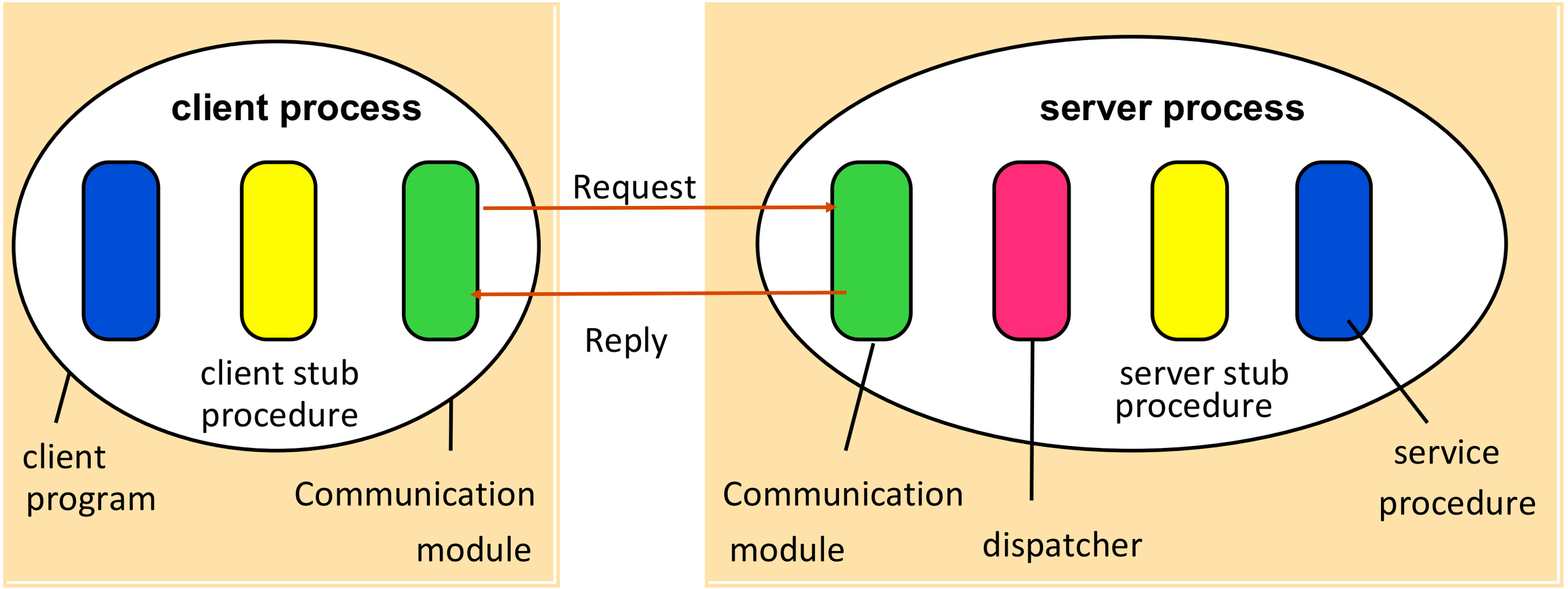
Remote Procedure Call (RPC)

- Implemented on top of Request-Reply protocol
- Basis of most client/server systems
- Simple – just like any procedure call
- Allows a client to call a procedure in a remote process
- Server can call another server, allowing chain of RPCs
- Procedural based
  - call a procedure which is located on a machine different from of the caller
- Parameters are passed during the call

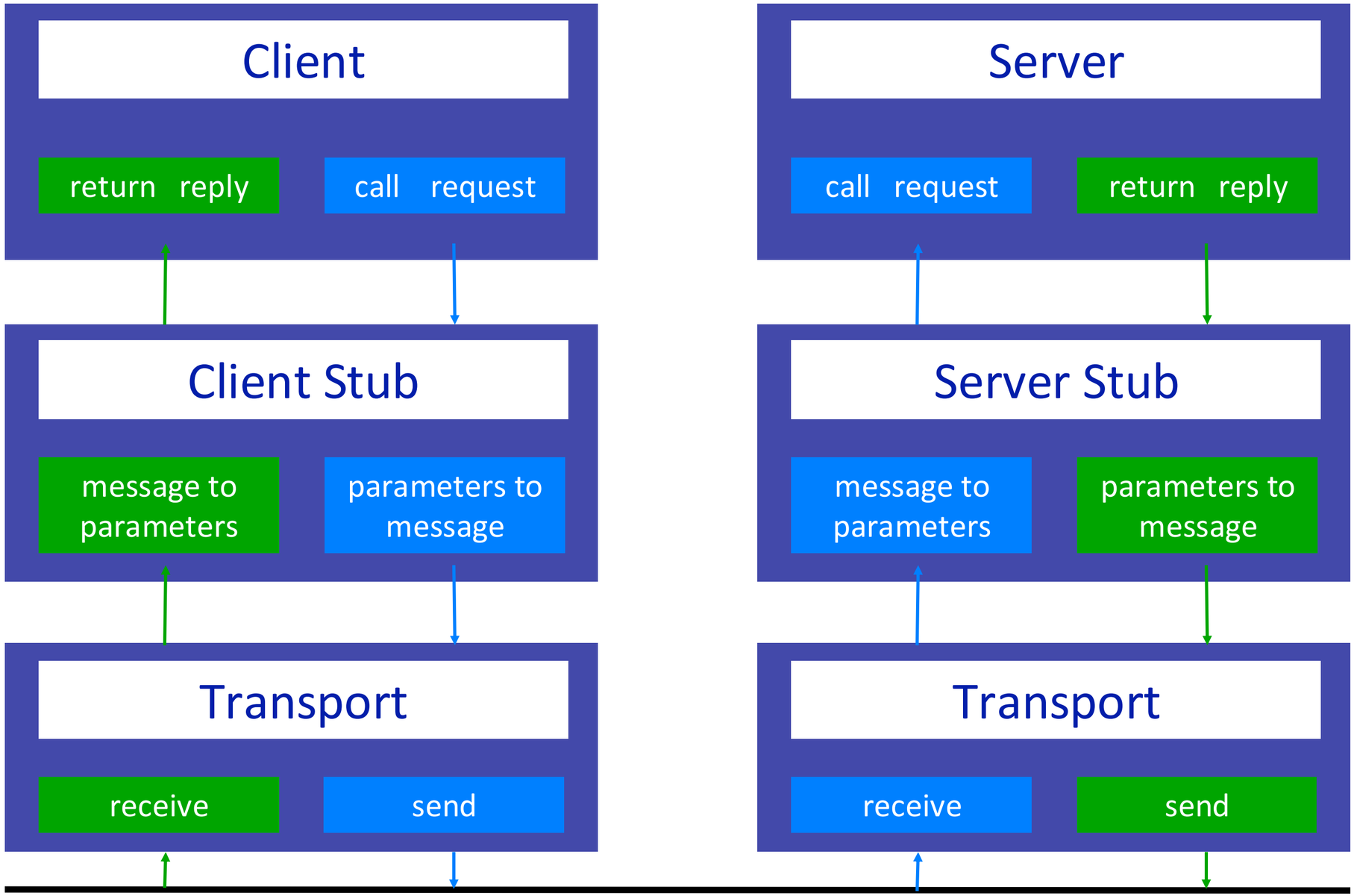
Remote Procedure Call (RPC) (cont.)

- Server defines interface of procedure that can call remotely
- Function of the stub:
  - Marshal the procedure id and the argument into a request message which is sent via a communication module to the server
  - Reply - un-marshal the results
- Stub – behave like a local procedure to client
- In the server, Dispatcher selects one of the server stub procedure

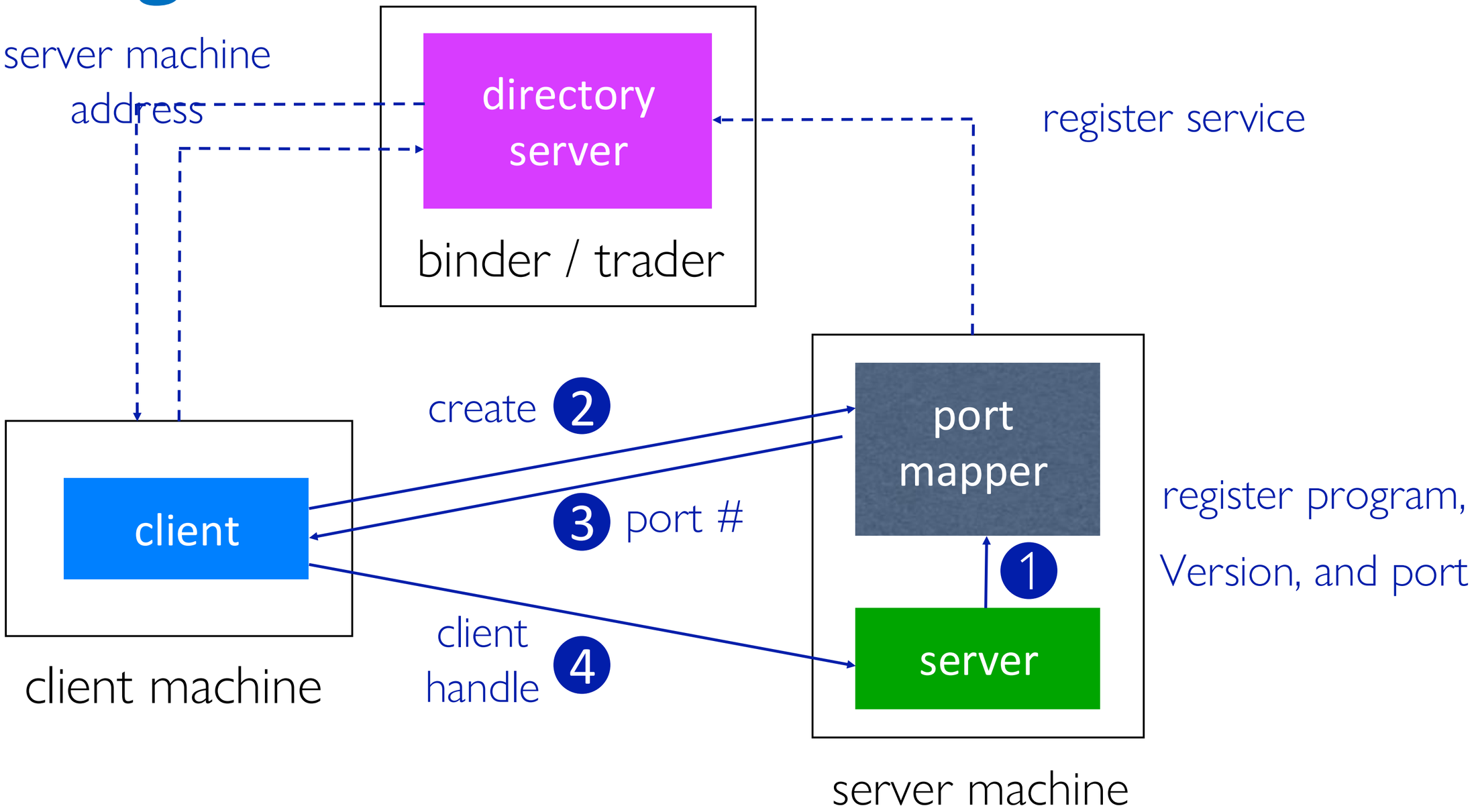
Client and Server in RPC



Flow of RPC


Client
return reply
call request
Client Stub
message to parameters
parameters to message
Transport
receive
send
Server
call request
return reply
Server Stub
message to parameters
parameters to message
Transport
receive
send

Binding between Client & Server

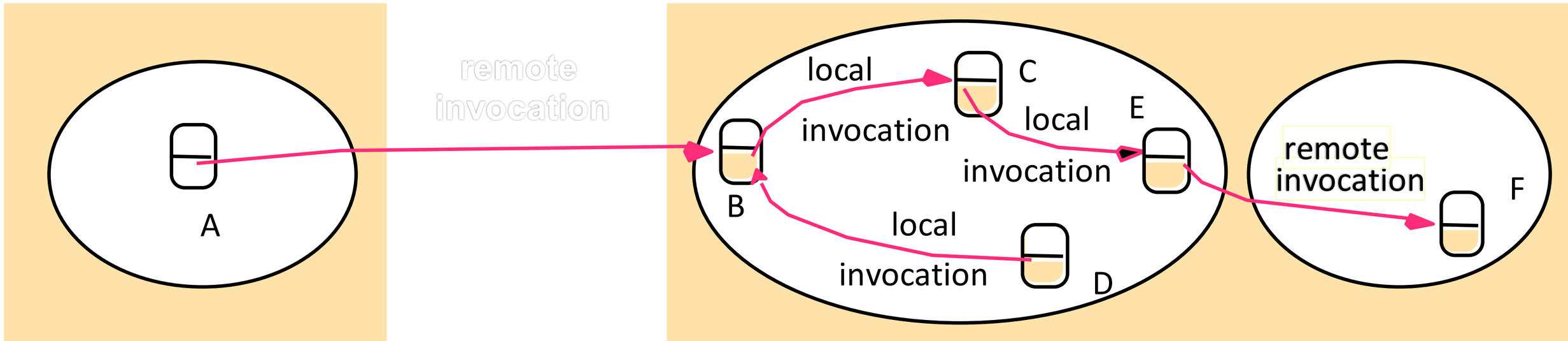


RPC Exchange Protocol

Name	*Messages sent by*		
	Client	*Server*	*Client*
R	*Request*		
RR	*Request*	*Reply*	
RRA	*Request*	*Reply*	*Acknowledge reply*

Remote Method Invocation (RMI)

- Method invocations between objects in different processes (within the same computer or not)



Remote and local method invocations

RMI (Implementation)

- Communication module carries out request-reply protocol
- Remote reference module translate between local and remote object references and create remote object references
- RMI software
  - Proxies – transparent by hiding the details of object reference, (un)marshalling arguments/results and send/receive message
  - dispatchers – select the appropriate method and pass to request message
  - Skeleton implements methods in remote interface

Summary

- What did I learn in this module?
  - The concept of processes and threads
  - Threads and distributed system
  - Virtualisation
  - Client-server model
  - Synchronous vs asynchronous
  - Socket
  - TCP vs UDP
  - RPC vs RMI

Acknowledgement

The figures and contents used in this lecture are adopted and adapted from:

- van Steen, M. and Tanenbaum A.S. (2023). Distributed Systems, 4th Edition. distributed-systems.net.
- Colouries, G., Dollimore, J., Kindberg, T and Blair, G. (2021). Distributed Systems: Concepts and Design, 5th Edition. Pearson.
- Thomas Erl , Zaigham Mahmood, and Ricardo Puttini (2013). Cloud Computing: Concepts, Technology & Architecture. Prentice Hall.
- Web references.

THANK YOU

Chee Sun LIEW
cheesunl@sunway.edu.my

